A TRANSACTION LOGGING APPLICATION FOR COURIER AND DOCUMENT SERVICE OUTLETS

BY

IMOKHAAI NATHAN OZEKHAI

B.SC COMPUTER SCIENCE

MATRIC NO: 22/10689

A RESEARCH PROPOSAL SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE, COLLEGE OF COMPUTING AND INFORMATION SCIENCES, CALEB UNIVERSITY, LAGOS IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF B.Sc. DEGREE IN COMPUTER SCIENCE.

MAY 2026

CERTIFICATION

This is to certify that this project report titled "**A TRANSACTION LOGGING APPLICATION FOR COURIER AND DOCUMENT SERVICE OUTLETS**" was carried out by **Imokhai Nathan Ozekhai** with Matriculation Number **22/10689** under the supervision of the Department of Computer Science, **Caleb University**, for the award of a **Bachelor of Science (B.Sc.)** degree in **Computer Science**.

Dr. OJO RASHEED _____

Supervisor's Name **Signature & Date**

Dr. AYORIDE .P. ODUROYE _____

Head Of Department's Name **Signature & Date**

DEDICATION

This work is dedicated to Almighty God for His infinite grace, wisdom, and strength which sustained me throughout this academic journey and brought this project to a successful completion.

It is also dedicated to my beloved parents, Mr. and Mrs. **IMOKHAI**, for their unwavering love, endless sacrifices, and constant support, both financially and morally, which laid the foundation for my academic success.

ACKNOWLEDGEMENT

First and foremost, I express my deepest gratitude to Almighty God for the gift of life, intellectual capacity, and guidance throughout my stay in this institution.

My sincere appreciation goes to my project supervisor, **DR. OJO RASHEED**, for his invaluable guidance, constructive feedback, and continuous encouragement from the inception to the completion of this software project.

I owe a debt of gratitude to my wonderful parents for their sacrifices and belief in my potential. Their prayers have been a constant pillar of support.

I would like to express my profound gratitude to **Ahube Chisom**, a data analyst whose impact on my technical growth has been monumental. Since our paths crossed during my Industrial Training while I was finding my feet in web development, he saw potential in me and brought me into his brand. His constant push for me to expand my technical horizons and master new stacks has not only driven my contributions to his upcoming startup, but has also inspired an ongoing desire in me to keep learning and striving for more. I am incredibly grateful for his mentorship, belief in my abilities, and his strategic advice on refining this project for the commercial market.

My heartfelt thanks also go to my dear friend, **Idaka Stephanie Eweli**, for being an incredible pillar of support during the most demanding periods of this journ ey. Whenever exhaustion set in and the stress felt overwhelming, her constant encouragement to keep pushing because I am almost at the finish line kept me from relenting. I am deeply grateful for her presence, her ability to lift my spirits, and the lighthearted moments that always helped ease my worries and clear my mind when I needed it most.

Finally, to everyone who contributed directly or indirectly to the success of this academic milestone, I say a profound thank you.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLE

ABSTRACT

This project presents the design and implementation of ApexLog Operation Hub, a unified transaction logging application developed to address the inefficiencies of fragmented tracking and manual record-keeping in multi-service business environments. Traditional operations across business centers that offer distinct services—such as document processing and courier logistics—frequently suffer from misplaced records, poor financial accountability, and lack of real-time package status visibility. To resolve these challenges, a centralized system was engineered using a modern technical stack comprising React, Next.js, TypeScript, and Tailwind CSS for a responsive, type-safe frontend, backed by a robust PHP and MySQL relational database structure. The system modules were categorized into customer profile management, document service logging (covering metrics like typesetting and print binding), and a dedicated courier logistics tracking component featuring unique tracking code generation. Object-Oriented Analysis and Design (OOAD) methodology was employed to model the system architecture, ensuring clear data abstraction and structured table relationships through primary and foreign keys. System testing demonstrated high performance in handling simultaneous write operations, instantaneous tracking status updates, and rigorous data integrity validation. The implementation of ApexLog Operation Hub effectively eliminates paper-based bottlenecks, streamlines workflows, and provides a scalable, commercially viable framework for multi-service operations.

Keywords: Transaction Logging, Multi-Service Environments, Next.js, Relational Database, Courier Tracking, ApexLog Operation Hub.

CHAPTER ONE

INTRODUCTION

1.1 BACKGROUND OF THE STUDY

The paradigms governing enterprise records management, data accounting, and transactional tracking have undergone profound structural transformations over recent decades, transitioning from labor-intensive analog frameworks to highly sophisticated, distributed computing ecosystems. Historically, small-to-medium enterprise (SME) organizations, particularly localized customer service centers, relied extensively on manual, paper-based record-keeping processes. These primitive methods were characterized by physical logbooks, manual ledger indexing, standard vertical filing cabinets, and localized paper archives. While these historical methodologies required negligible initial capital expenditure and no specialized technical literacy, they presented severe operational bottlenecks that severely constrained enterprise throughput. Among these challenges were significant processing inefficiencies, high data vulnerability to physical degradation or environmental hazards, a high susceptibility to human clerical error, and an absolute lack of horizontal or vertical scalability when handling elevated daily transaction volumes.

With the rapid progression of microprocessing capabilities, affordable storage media, and local area network (SME) hardware infrastructure, semi-automated record management systems emerged as an intermediate technological bridge. This architectural shift allowed enterprises to digitize physical records using relational spreadsheet software and basic desktop-bound standalone database management applications. These intermediate digital frameworks substantially improved structural data organization, accelerated basic localized search times, and minimized the total risk of catastrophic data loss resulting from physical damage or paper decay. However, these early digital architectures remained fundamentally disjointed and localized. Data repositories were routinely siloed across isolated computing

terminals and conflicting operational platforms, which severely inhibited unified cross-functional operations, multi-layered data auditing, and real-time administrative transparency.

In contemporary software engineering environments, fully automated systems such as Document Management Systems (DMS), Enterprise Resource Planning (ERP) enterprise suites, and dedicated Courier Management Systems (CMS) have been engineered to streamline complex business operations. These modern architectures leverage enterprise-grade technologies—including cloud computing environments, microservices, headless web-based software delivery frameworks, and sophisticated relational database management systems (RDBMS)—to optimize data persistence, minimize computational query retrieval latencies, and handle concurrent multi-user transaction processing safely. Consequently, they provide superior system execution efficiency, elastic scalability, and multi-tenant data accessibility when compared to previous tracking paradigms.

Despite these major technological milestones, a critical structural gap persists in the contemporary commercial software ecosystem. The vast majority of modern operational logging software packages are explicitly tailored, priced, and scaled for large-scale enterprise organizations that exhibit highly complex, sprawling multi-departmental workflow configurations. As a direct consequence of this design philosophy, such platforms are financially cost-prohibitive due to recurring Software-as-a-Service (SaaS) licensing fees, computationally over-engineered with unnecessary feature bloat, and fundamentally unsuited for small-scale, localized multi-service outlets operating under stringent resource constraints. Furthermore, current commercial solutions are heavily siloed into isolated use cases—providing either standalone secure document storage or independent logistics/fleet management tracking—but fail to provide an integrated transaction logging framework

capable of capturing multiple distinct service workflows (such as simultaneous courier shipments and document processing orders) within a singular, unified application interface.

To address this distinct operational challenge while adhering to strict software adoption principles, this study is theoretically grounded in the Technology Acceptance Model (TAM). Originally formulated to model user behavior, TAM postulates that user adoption and behavioral intention to utilize a newly introduced system are primary functions of two core variables: Perceived Usefulness (PU) and Perceived Ease of Use (PEOU). Within the daily operational context of a small business service center, a software application's practical long-term adoption relies directly on interface responsiveness, structural simplicity, minimal client-side computational overhead, execution efficiency, and immediate functional alignment with localized workflows.

To maximize these TAM variables, modern software design increasingly favors a decoupled or headless architecture pattern. This design pattern structurally isolates the user interface presentation layer from the underlying server-side application logic and database engine.

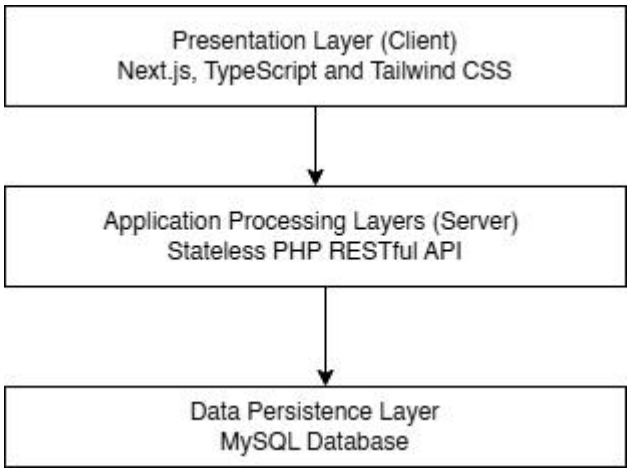

Figure 1.1: Architectural Block Diagram of the Presentation, Application Processing, and Data Persistence Layers

Therefore, the system proposed in this research project is engineered using a decoupled multi-tier web paradigm. It leverages a component-driven, highly optimized frontend

framework built using Next.js, TypeScript, and Tailwind CSS to maximize Perceived Ease of Use through sub-second interface rendering. Concurrently, it implements a lightweight, stateless backend engine utilizing PHP and MySQL to maximize Perceived Usefulness by offering an efficient, integrated tracking ledger tailored specifically for localized document and courier outlets.

1.2 STATEMENT OF THE PROBLEM

The development and implementation of a unified transaction logging platform are necessitated by several critical technical, financial, and structural deficiencies characterizing localized multi-service operational environments:

1.2.1 Vulnerability and Structural Integrity Failures of Manual Record-Keeping

Localized multi-service business environments continue to exhibit an extensive reliance on manual, analog record-keeping systems. This dependency manifests as the daily utilization of physical paper notebooks, loose paper receipts, and unstructured filing folders to capture and preserve vital transaction records. The operational impact of this approach is a severe vulnerability to structural data loss through physical liquid spills, fire hazards, and general paper degradation over time. Furthermore, clerical input errors are common, and the manual modification of historical logs makes data verification difficult, causing regular structural integrity failures within the business ledger.

1.2.2 High Computational Latency and Operational Friction in Record Retrieval

The absence of digital indexing structures in local service outlets causes high operational latencies during customer query processing. This problem manifests as administrative staff spending excessive time searching chronologically through historical paper logs or un-indexed flat files to locate a specific past customer profile, service history row, or courier tracking code. The resulting operational impact is an inflation of processing cycle times,

significant customer queues, lowered daily transaction throughput, and heightened cognitive friction for the system operators.

1.2.3 Data Fragmentation Across Disjointed Multi-Service Workflows

A significant challenge within modern small-scale service centers is the complete lack of functional software integration between distinct commercial service offerings. This manifests as the detached handling of document processing workflows (such as digital typesetting, large-scale printing, scanning, and laminating) and courier logistics transactions (such as package registration, destination zoning, and dispatch indexing) on separate, non-communicating tracking mediums. The operational impact is a highly fragmented data layer, which forces staff into redundant data entries, prevents unified daily auditing, and limits the owner's ability to track overall business health.

1.2.4 Architectural Over-Engineering and Prohibitive Licensing Barriers

Small-scale service outlets face severe accessibility barriers when attempting to adopt existing commercial information management software. This barrier manifests as the structural exclusion of small businesses from modern enterprise markets due to cost-prohibitive upfront or subscription-based SaaS licensing fees, complex cloud infrastructure prerequisites, and rigid, over-engineered system configurations that cannot be customized to fit lean operations. The long-term impact is a persistently low adoption rate of modern digital tools among SMEs, forcing them to remain dependent on manual processes (Jordan et al., 2022).

1.2.5 Absence of Real-Time Operational Analytics and Reporting Mechanisms

The final core deficiency is the structural lack of automated data summarization and financial reporting frameworks tailored for local business owners. This manifests as an inability to automatically compile, filter, and extract accurate operational summaries, daily transaction counts, or periodic revenue statements. The downstream impact is that managers face severe

difficulties when attempting to evaluate staff productivity, audit financial leakages, assess shifting service demands, or make data-driven decisions to optimize business growth (Cherchata et al., 2022).

1.3 SIGNIFICANCE OF THE STUDY

This research provides both immediate practical value to small business environments and distinct academic contributions to the broader domain of Information Management Systems (IMS).

1.3.1 Practical Significance for Service Outlets and Owners

- Decoupled Data Centralization: By substituting fragmented paper ledgers with a unified, relational database structure, the application establishes a single source of truth. This eliminates data scattering and prevents redundant customer records across different service workflows.
- Mitigation of Input Error and Query Latencies: Implementing an advanced user interface with strict compile-time validation rules minimizes the entry of malformed data fields. Concurrently, database indexing minimizes search query responses to sub-second thresholds, boosting overall customer processing speeds.
- On-Demand Operational Business Intelligence: The platform empowers administrative staff and business owners to instantly compile and view real-time data summaries and chronological financial reports, providing accurate operational analytics to support strategic business decisions.
- Interface Unification and Cognitive Load Reduction: The system unifies independent workflows under a modern, single-page application dashboard. This reduces cognitive fatigue for operators, minimizes application context switching, and significantly accelerates worker onboarding speeds.

1.3.2 Academic Significance to Computing Literature

- Architectural Reference for Decoupled Headless Implementations: From a computer science perspective, this project provides a clear technical blueprint for applying a decoupled architectural pattern (Next.js client layer separated from a stateless PHP RESTful API engine) to resolve practical operational challenges in small enterprises.
- Addressing Literature Gaps in Small-Scale Multi-Service Frameworks: This research enriches computing literature by explicitly focusing on integrated, lightweight transaction logging architectures for multi-service environments—a specific niche that has historically received sparse academic focus compared to large-scale enterprise ERP systems.
- Typographic and Compile-Time Verification Blueprint: This study establishes an academic baseline for implementing type-safe data schemas on the client layer using TypeScript. This demonstrates how to eliminate interface-to-API type mismatches and handle asynchronous communication cleanly before transferring records to relational database daemons.

1.4 AIM AND OBJECTIVES OF THE STUDY

1.4.1 Aim of the Study

The primary overarching aim of this research project is to design, implement, and evaluate a lightweight, decoupled multi-tier transaction logging software application engineered for the centralized management of customer records and distinct service transactions within integrated courier and document service outlets.

1.4.2 Objectives of the Study

To successfully realize this broad aim, the following specific technical and analytical objectives must be achieved:

- To critically analyze the daily operational workflows, system requirements, data flow diagrams, and bottlenecks of existing manual and semi-automated transaction recording methods within local multi-service centers.
- To design a structured relational data model and a decoupled system architecture capable of managing unified customer profile identities and multi-service transaction records without data duplication.
- To implement an interactive, type-safe, component-driven client user interface using Next.js, TypeScript, and Tailwind CSS to log document processing and courier service details smoothly.
- To implement and optimize a localized PHP RESTful API engine backed by a MySQL database daemon for secure data persistence, fast row indexing, and high-speed asynchronous data parsing.
- To systematically test, debug, and evaluate the completed application framework against strict usability, type compliance, and functional data validation metrics under simulated operational workloads.

.

1.5 METHODOLOGY

This software engineering project adopts the classic Waterfall Software Development Lifecycle (SDLC) Methodology, applying a highly structured, sequential framework to guide the system from initial requirement formulation to final localized deployment. The linear progression of the Waterfall model provides a stable development framework, facilitating complete requirement definitions, structural schema lock-in, and strict architectural modeling before any source code is generated.

1.5.1 Requirement Analysis Phase

This phase involves identifying and documenting detailed operational expectations regarding multi-service operations. It maps user entry workflows, fields for courier logistics, and categories for document processing jobs. The outputs define the strict functional specification parameters (such as form schema validations) and non-functional metrics (such as API fetch latency thresholds, component load times, and responsive layout breakpoints across device screens).

1.5.2 System Design Phase

The collected requirements are translated into a comprehensive technical architecture layout. Semantic modeling tools, including Entity Relationship Diagrams (ERD), Data Flow Diagrams (DFDs), and logical System Flowcharts, are leveraged to map MySQL database cardinalities, secure primary-foreign key relationships, and chart the asynchronous HTTP request-response pipelines between the isolated frontend UI components and the backend data tier.

1.5.3 Implementation Phase

This phase marks the translation of structural models and architectural designs into clean, executable source code using a modern, decoupled web application stack:

- Frontend Presentation Layer: Engineered within a Next.js framework using TypeScript to achieve a highly responsive Single Page Application (SPA) environment. Type definitions ensure strict structural data compliance before payload transmission. Styling is handled via utility primitives in Tailwind CSS to optimize asset delivery and UI rendering speeds.
- Backend Application Logic Layer: Programmed as a stateless PHP RESTful API engine. This tier governs incoming asynchronous HTTP requests, processes server-side business

validation loops, sets necessary Cross-Origin Resource Sharing (CORS) headers, and returns normalized JSON data payloads.

- Database Persistence Layer: Realized via a relational MySQL database configured within a local environment to handle transactional data tables, primary-foreign key relationships, and index-accelerated query matching.

1.5.4 Testing Phase

The completed decoupled framework is subjected to multi-tiered quality assurance protocols to guarantee long-term stability. This includes Unit Testing to validate isolated PHP endpoint scripts, Type Checking via the TypeScript compiler (tsc) to eliminate interface-to-API data layout mismatches, Integration Testing to confirm faultless JSON data transfers between Next.js and XAMPP/Apache, and User Acceptance Testing (UAT) to verify user satisfaction and ease of use.

1.5.5 Deployment Phase

The completed system is deployed into a localized server environment. The PHP backend API and MySQL database run via the XAMPP stack (Apache HTTP Server), while the Next.js client application runs via a localized client runtime container. This setup provides a cost-effective, high-performance offline architecture tailored perfectly for local business IT infrastructure.

1.6 SCOPE AND LIMITATIONS OF THE PROJECT

1.6.1 PROJECT SCOPE

The functional boundaries and operational features of this transaction logging application are limited to the following capabilities:

- The development of a functional transaction logging platform optimized for multi-service environments using a decoupled web architecture.

- A Customer Registration and Profile Management Module: Built using reusable React components to collect, validate, and dynamically search unique customer profiles.
- A Document Services Logging Module: Interfaces designed to capture and categorize business center transactions, including digital typing, large-scale printing, document binding, and scanning.
- A Courier Logistics Logging Module: Forms optimized to record sender identities, receiver criteria, shipping metrics, package weight classifications, and generate localized tracking indices.
- An Asynchronous Search & Filter Algorithm: Utilizing native JavaScript fetch routines communicating with backend SQL query indexes to parse and return specific transactions without refreshing the page.
- An Automated Operational Summary Module: A client dashboard interface that processes JSON response metrics to render daily and periodic transaction summaries.
- Deployment within a localized internal web network (Apache/XAMPP environment) without relying on live external cloud servers.

1.6.2 PROJECT LIMITATION

To manage system evaluation criteria during the defense, the following features are explicitly excluded from this system version:

- Real-time automated tracking synchronization with external global shipping APIs (e.g., DHL, FedEx).
- Production deployment onto global multi-server cloud clustering architectures (e.g., AWS, Google Cloud Platform).
- Advanced security mechanisms like hardware-based Multi-Factor Authentication (MFA) or field-level data layer encryption keys.

- Native mobile application development for mobile operating systems (iOS or Android).
- Live integration with external electronic payment gateways or digital financial clearing houses.
- Multi-tenant distributed database synchronization across separate geographical locations.

1.7 ORGANIZATION OF THE WORK

This research project is organized into five interconnected chapters presenting a logical progression from theory to execution:

Chapter One establishes the introductory context, covering the background of the study, the specific problem statement, study significance, research objectives, scope boundaries, methodology summary, and structural limitations.

Chapter Two explores the literature review, performing a granular analysis of existing document storage frameworks, logistics engines, and decoupled web architectures to highlight the technical gap our project solves.

Chapter Three details the complete system analysis and design phase, specifying functional system engineering requirements, ERD relational database data schemas, API payload structures, and the decoupled 3-tier architectural layout.

Chapter Four focuses entirely on system implementation mechanics, describing Next.js component configurations, PHP API controllers, Tailwind CSS layouts, and structural test case metrics.

Chapter Five concludes the work by offering a comprehensive summary, project conclusions, and concrete recommendations for future technological migrations.

CHAPTER TWO

LITERATURE REVIEW

2.1 INTRODUCTION AND HISTORICAL REVIEW OF THE PROBLEM

The systemic paradigms governing business record administration, information retrieval, and operational data persistence have undergone a major architectural evolution over the past several decades (Sommerville, 2021). This structural transformation has been directly catalyzed by rapid advancements in computer engineering, relational database design, and enterprise web technologies (Silberschatz et al., 2020).

Historically, small-to-medium business service outlets operated within a completely manual framework (Afaishat et al., 2024). This primitive era was characterized by an absolute reliance on physical, paper-bound mediums—including ledger notebooks, unindexed paper receipts, manual filing cabinets, and handwritten logbooks—to record vital customer profiles and transactional data (Hayati et al., 2020). Although these manual methods offered low economic entry barriers and required zero technical literacy from counter personnel, they introduced severe operational risks (Laudon & Laudon, 2021). These vulnerabilities included high susceptibility to catastrophic physical data loss via environmental hazards such as fire or moisture, long retrieval latency times, a high incidence of human data-entry errors, and a total lack of structural scalability when handling concurrent user requests or elevated transaction volumes (Stair & Reynolds, 2020).

With the subsequent proliferation of microcomputers and localized workstation hardware, business operations shifted toward a semi-automated paradigm (Wibowo, 2023). This transition was marked by the introduction of standalone computing terminals running rudimentary, localized spreadsheet programs and primitive, non-relational database management software (Coronel & Morris, 2023). While these digital spreadsheets improved record legibility and minimized physical storage footprints, they introduced deep

architectural fragmentation. Data repositories remained completely isolated across disconnected storage nodes and separate folders, creating severe structural synchronization barriers (Silberschatz et al., 2020). Because these frameworks lacked a unified relational database engine, centralized access control, or indexed search algorithms, cross-referencing information or tracking a single customer's journey across multiple service streams remained computationally and operationally inefficient (Elmasri & Navathe, 2021).

The subsequent convergence of modern Database Management Systems (DBMS), object-relational mapping, and multi-tier web architectures introduced the contemporary era of fully automated computing ecosystems (Pressman & Maxim, 2020). This technology wave yielded highly sophisticated platforms, including Electronic Document Management Systems (EDMS), automated courier routing suites, and Enterprise Resource Planning (ERP) applications (Davenport, 2021). Modern information systems utilize advanced frameworks—such as relational cloud architectures, server-side data validation pipelines, and ubiquitous web-browser interfaces—to drastically optimize data persistence security, execution speeds, and concurrent operational capabilities (Richardson et al., 2022; Dynamic Web Solutions, 2022).

Despite these massive technological breakthroughs, a profound architectural gap remains unaddressed within the contemporary software market (Ikhwan & Himawati, 2024). The vast majority of commercially available software solutions are explicitly engineered for large-scale enterprise environments characterized by sprawling, complex organizational structures (Triandini et al., 2023). As a direct consequence of this design philosophy, current market solutions are financially cost-prohibitive, demand high hardware overhead, require specialized IT personnel for deployment, and are functionally over-engineered for small business operations (Astuty et al., 2024).

More importantly, the software landscape remains heavily siloed: existing platforms are rigidly split into specialized, standalone engines that handle either document processing archives or logistics tracking networks independently (Parviainen et al., 2022). There is a distinct absence of integrated, lightweight, and unified architectures capable of consolidating multiple distinct service transaction lines into a singular, cohesive database dashboard (Hendricks & Mwapwele, 2023). This systemic deficiency forces small-scale multi-service hubs to either purchase multiple expensive software subscriptions or revert to legacy manual filing methods, creating a clear need for a localized, integrated transaction logging application (Dynamic Web Solutions, 2022).

2.2 SYSTEMATIC REVIEW OF EXISTING SOLUTIONS

To contextualize the development of the proposed integrated system, it is vital to evaluate the technical architectures, methodologies, and operational scopes of solutions previously engineered by researchers within the domains of document management, electronic archiving, and courier logistics.

2.2.1 Document Management and Electronic Archiving Systems

Nagrama et al. (2024) designed and engineered a web-based document management system explicitly optimized to streamline the lifecycle of administrative office documents. The system leveraged web interfaces to facilitate the electronic upload, indexing, retrieval, and downloading of digital assets, proving that paper-based storage is highly inefficient for modern organizational workflows. While their application successfully improved digital asset accessibility and document security, its underlying data model was built strictly for file parsing and static storage. It lacks any transactional processing engines, customer relationship tables, or integrated courier logging hooks, making it functionally incapable of driving multi-service commercial workflows.

Alade (2023) proposed and evaluated a web-based document management framework tailored for institutional record-keeping. The system focused on minimizing the physical constraints of manual file handling by automating storage mapping and optimizing directory indexing within an automated software environment. Although Alade's research successfully demonstrated that electronic document management drastically enhances workflow throughput, the system's backend logic was restricted to document storage. It lacked the necessary application processing instances to log real-time financial service transactions or track external service dispatches.

Pradana et al. (2022) developed an operational digitalization framework focused on converting highly vulnerable paper documents and manual organizational processes into electronic formats to ensure long-term preservation and efficiency. The study highlighted the specific structural degradation risks associated with physical paper logs. However, the system was architected primarily as a static, read-heavy digital archive rather than a dynamic, write-heavy transaction logging platform. It provided no structural provisions for real-time customer data entry or operational service tracking at a commercial point-of-sale.

Hofisi and Chigova (2023) designed a digital transformation strategy aimed at mitigating workflow bottlenecks and boosting workplace productivity through high-speed digital indexing and retrieval routines. While their system minimized file search latencies, the underlying database schema was engineered exclusively for document metadata and organizational structuring. It offered no functional modules or relational tables to handle courier logistics, sender-receiver tracking matrices, or multi-tier business operations.

Ogunleye et al. (2020) conducted a deep architectural analysis regarding the systemic transition from paper archiving methods to electronic document management systems within modern enterprises, establishing strict digital preservation and archival compliance standards. While their study provided foundational literature regarding the operational benefits of digital

record-keeping, the research remained theoretical and highly focused on archival standards. It failed to proffer a functional, real-time transaction logging codebase suitable for the rapid, day-to-day operational realities of small service outlets.

Han et al. (2020) proposed a cloud-based electronic document management system utilizing cloud storage infrastructures to deliver scalable file sharing and fine-grained, role-based user access controls. The deployment of cloud resources substantially optimized multi-user collaboration and storage elasticity. However, this enterprise architecture demands consistent high-speed internet connectivity and introduces recurring subscription costs, rendering it completely unsuited for local, small-scale businesses that require low-cost, offline-capable, locally hosted solutions like a XAMPP-driven Apache environment.

Jordan et al. (2022) explored the structural impact of modern document management systems as core drivers of digital transformation and workflow automation within corporate organizations. The authors verified that migrating to digital document management significantly cuts paper costs and improves overall operational efficiency. However, their research focused heavily on macro-level corporate digital transformation strategies rather than addressing the micro-level, practical programming needs of small business centers requiring a unified point-of-sale logging mechanism.

2.2.2 Courier Management and Logistics Tracking Frameworks

Nel and Masilela (2020) engineered a framework for integrating automated tracking systems designed to digitize parcel booking, transit tracking, staff dispatch allocation, and shipment record preservation. Their system successfully reduced manual paper dependencies within logistics workflows and enhanced operational accuracy via advanced tracking integrations. Nevertheless, the system's software architecture was built exclusively for logistics routing; it possessed no underlying data definitions, user forms, or processing modules to handle localized document services like typing, printing, or digital scanning logs.

Cherchata et al. (2022) examined corporate innovations within logistics management, focusing on optimizing material and information flows through enterprise supply chains. While their study underscored the strategic value of crisp information routing, the research was highly conceptual and geared toward large supply chain enterprises. It did not present a lightweight, deployable software application capable of executing localized transaction logging for a small, independent service counter.

2.2.3 Service-Oriented Frameworks

Fan et al. (2022) proposed an advanced service-oriented framework engineered to track runtime consumer behavioral patterns and digital interactions across multiple operational layers to tailor digital tools effectively. While their research contributed valuable insights into multi-tiered system stability, customer interaction management, and logic verification, their framework operated at a highly abstract middleware layer. It did not provide a practical application interface for front-counter business activities, customer account creation, or everyday transaction logging.

2.3 CURRENT TRENDS OF SOLUTIONS

Modern trends within the domain of Information Management Systems (IMS) are heavily defined by full workflow automation, digital asset persistence, cloud computing integration, and responsive web-based service delivery. Current software engineering methodologies favor a complete departure from manual, paper-bound business logs in favor of cross-platform web applications that maximize data processing velocities and minimize retrieval latency.

Architecturally, modern management systems are built using decoupled, multi-tier frameworks. The frontend layer heavily utilizes HTML5, CSS3, and modern JavaScript engines to generate highly intuitive, responsive User Interfaces (UIs) that minimize user

errors and accelerate data entry. Concurrently, the backend application layer leverages robust server-side scripting languages like PHP, combined with indexed relational database engines like MySQL, to ensure secure data persistence, relational data integrity, and high-speed query execution.

Furthermore, a significant market trend involves the integration of historically fragmented business operations into a unified software environment. Rather than forcing an enterprise to maintain separate, siloed software applications for distinct tasks, contemporary system designs prioritize consolidating customer relationship management (CRM), transaction recording, audit logs, and analytical reporting into a singular master dashboard.

However, despite this clear trend toward systems integration, commercial developers continue to overlook small-scale enterprises. Existing integrated solutions are dominated by massive, expensive, cloud-hosted ERP platforms tailored for large multi-national corporations. Consequently, small multi-service outlets that simultaneously provide document services and courier logistics remain caught in a technical divide, lacking a localized, affordable, and lightweight platform customized to their precise multi-service requirements.

2.4 PROPOSED AREA OF IMPROVEMENT FROM LITERATURE ANALYSIS

The comprehensive body of literature reviewed clearly indicates that existing software engineering efforts provide robust solutions for document management, electronic archiving, courier tracking, and global logistics operations as independent, isolated domains. However, a critical academic and practical gap persists: most of these platforms are explicitly engineered for single-industry configurations and fail to provide a unified architecture for multi-service business settings.

Therefore, the gap this project fills is the critical lack of a unified, low-cost, and simplified transaction logging framework specifically tailored for small-to-medium multi-service outlets.

While existing literature extensively documents robust electronic archiving systems and specialized courier tracking logistics, current market trends favor highly complex enterprise solutions that remain financially and operationally out of reach for small business environments. There is a distinct absence of a singular platform that integrates both document handling logs and courier service details within a localized, lightweight architecture.

This research directly addresses this void by consolidating customer registration, document service logging, and courier transaction tracking into a single, cohesive, web-based platform. By replacing fragmented, manual, or cost-prohibitive options with a highly practical, three-tier local web application, this study bridges the divide between theoretical database implementation and the real-world operational survival of small-scale multi-service businesses.

Unlike the complex enterprise software architectures reviewed in the literature, the system proposed in this study prioritizes structural simplicity, low computational overhead, and zero dependence on continuous external internet connectivity by utilizing a localized Apache web server environment via XAMPP. This ensures that small business owners can eliminate operational latency, prevent catastrophic paper data loss, and maintain real-time audit control over their multi-service businesses within an affordable framework.

CHAPTER THREE

METHODOLOGY

3.1 PROBLEM STATEMENT AND METHODS OF SOLUTION

Within contemporary micro-business ecosystems, local retail service centers providing low-margin, high-volume operations—such as document-centric workflows (digital typesetting, large-format printing, high-speed document scanning, binding, and laminating) alongside third-party or localized courier logistics tracking—face a profound operational paradox. While large enterprise logistics and document outsourcing organizations utilize sweeping, cloud-native Enterprise Resource Planning (ERP) applications to synchronize disparate service vectors, small-scale multi-service retail hubs remain deeply entrenched in archaic, inefficient operational environments. These businesses persistently rely on legacy manual tracking methods, which are structurally manifested through the utilization of physical paper ledger notebooks, loose thermal paper receipts, unindexed standalone spreadsheet applications, and deeply nested, unorganized local operating system directories.

This total reliance on decentralized and manual record compilation introduces severe systemic risks and immense processing overhead. Firstly, physical paper registries present an immediate, non-mitigated threat of catastrophic business data asset destruction via physical wear, liquid exposure, rodent damage, or fire hazards, while offering zero access control matrices, data masking, or audit transparency. Secondly, from an information retrieval perspective, execution latencies scale linearly ($O(N)$ time complexity) with the physical volume of daily entries. Front-counter operators must exhaustively and chronologically scan paper pages or unrelated static spreadsheets to isolate historical customer identities, past service history rows, or package delivery states. This causes major operational delays, reduced transactional throughput, and degraded customer experiences. Finally, because data for document services and courier entries are captured completely out of band from one

another, systemic data duplication, fragmented operational reporting, and a total breakdown of aggregate business performance monitoring occur.

To systematically mitigate these operational and engineering challenges, this study adopts a structured, object-oriented system development and engineering lifecycle framework grounded in decoupled architecture principles. The proposed method of solution entails the design, engineering, and implementation of a lightweight, highly responsive, decoupled multi-tier web transaction logging application running a centralized relational storage model. By mapping business operations into deterministic software rules and bounded relational tables, this platform automates customer identity management, eliminates data redundancy, indexes operational receipts, and exposes real-time analytical aggregation tools to the business administration.

The technical execution of this engineering solution strictly maps to a formal, sequential Software Development Lifecycle (SDLC) pipeline spanning rigorous Requirement Analysis, Structural Database and API Endpoint Design, Implementation via a modern decoupled scripting and framework stack (Next.js/TypeScript/Tailwind CSS client-side and Stateless PHP/MySQL server-side), Multi-layered Test Engineering, and Localized Containerized Network Deployment.

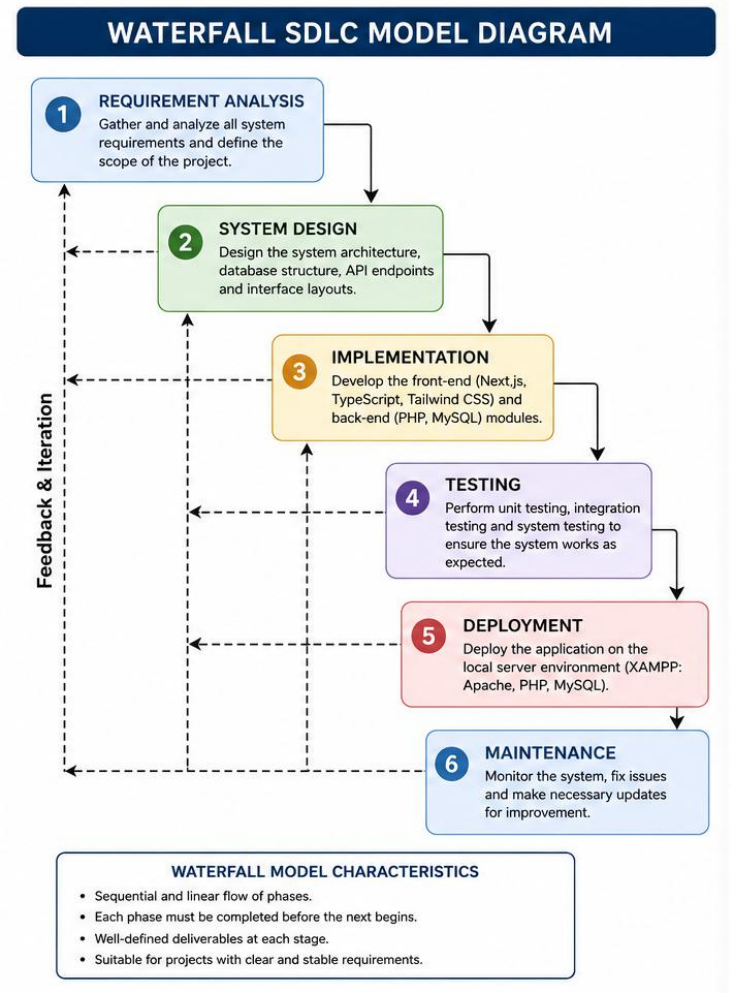

Figure 3.1: Waterfall SDLC Model Diagram for ApexLog Operation Hub Core Implementation. Source: Compiled by the Researcher (2026).

3.2 THE SOLUTION ALGORITHMS CONCEPT

The client-facing and server-side application layers of the proposed system are strictly governed by a collection of deterministic, finite solution algorithms. These algorithms explicitly outline the execution paths, input constraints, computational logic loops, type-guards, and database-state transitions managed by the software runtime environment. By mapping operations to formal algorithms, the software guarantees that all data committed to

the underlying persistence engine is thoroughly sanitized, validated, and tightly bounded by relational database integrity constraints.

3.2.1 Solution Algorithm 1 (Customer Registration Algorithm)

The system's customer registration sequence is explicitly designed to establish an isolated identity record within the relational data model prior to logging any commercial service interactions. This algorithm handles string capture, multi-field client-side type assertion checks, asynchronous HTTP POST payloads, and execution of server-side data definition and manipulation commands.

Formal Algorithmic Steps:

- Step 1: Start – Initialize the customer creation routine when triggered by an authenticated administrative user via the browser user interface event listener.
- Step 2: Render Client-Side Input Form – The Next.js engine renders the React component containing specialized HTML5 text input nodes, strictly bounded by client-side properties including string length bounds, numeric regular expressions, and required field validation rules.
- Step 3: Collect Input Attributes – The operator enters primary customer metadata properties, explicitly comprising firstName: string, lastName: string, phoneNumber: string, and emailAddress: string.
- Step 4: Execute Client-Side Type Guarding & Client-Side Validation – The TypeScript engine evaluates the form state variables against defined interface contracts, verifying that mandatory values are non-empty strings, alphabetical arrays conform to naming standards, telephone fields match regional digit lengths, and email structures satisfy RFC 5322 string pattern matching rules.
- Step 5: Process Validation Failures Conditional Branch – If client-side validation evaluates to false:

- Halt the payload dispatch routine.
- Maintain active volatile memory component state.
- Map context-specific string errors to the user interface fields using state arrays.
- Redirect control back to Step 2.

- Step 6: Asynchronous Payload Transmission – If validation evaluates to true, wrap the form state into a JSON payload. Use the JavaScript native asynchronous fetch() API to dispatch an HTTP POST request to the targeted server endpoint: http://localhost/api/v1/customers/create.php.
- Step 7: Server-Side Interception and Sanitation – The background PHP thread intercepts the request, decodes the raw input stream, applies SQL injection filtering and regular expression sanitization, and opens a secure connection to the database.
- Step 8: Storage Layer Injection and Verification – Construct a parameterized, sanitized SQL INSERT statement targeting the customers table. The storage subsystem assigns an auto-incremented primary key (Customer_ID), writes the row to persistent storage, and returns an HTTP status code 201 Created containing a JSON success confirmation block back to the client interface.
- Step 9: Handle Success State Presentation – Upon receiving the status code 201, the Next.js client component updates its state layout, resets form entry fields, clears error logs, and displays an on-screen notification confirming safe registration.
- Step 10: End – Terminate the customer registration thread and return the presentation layer to an active listening state, waiting for subsequent operational commands.

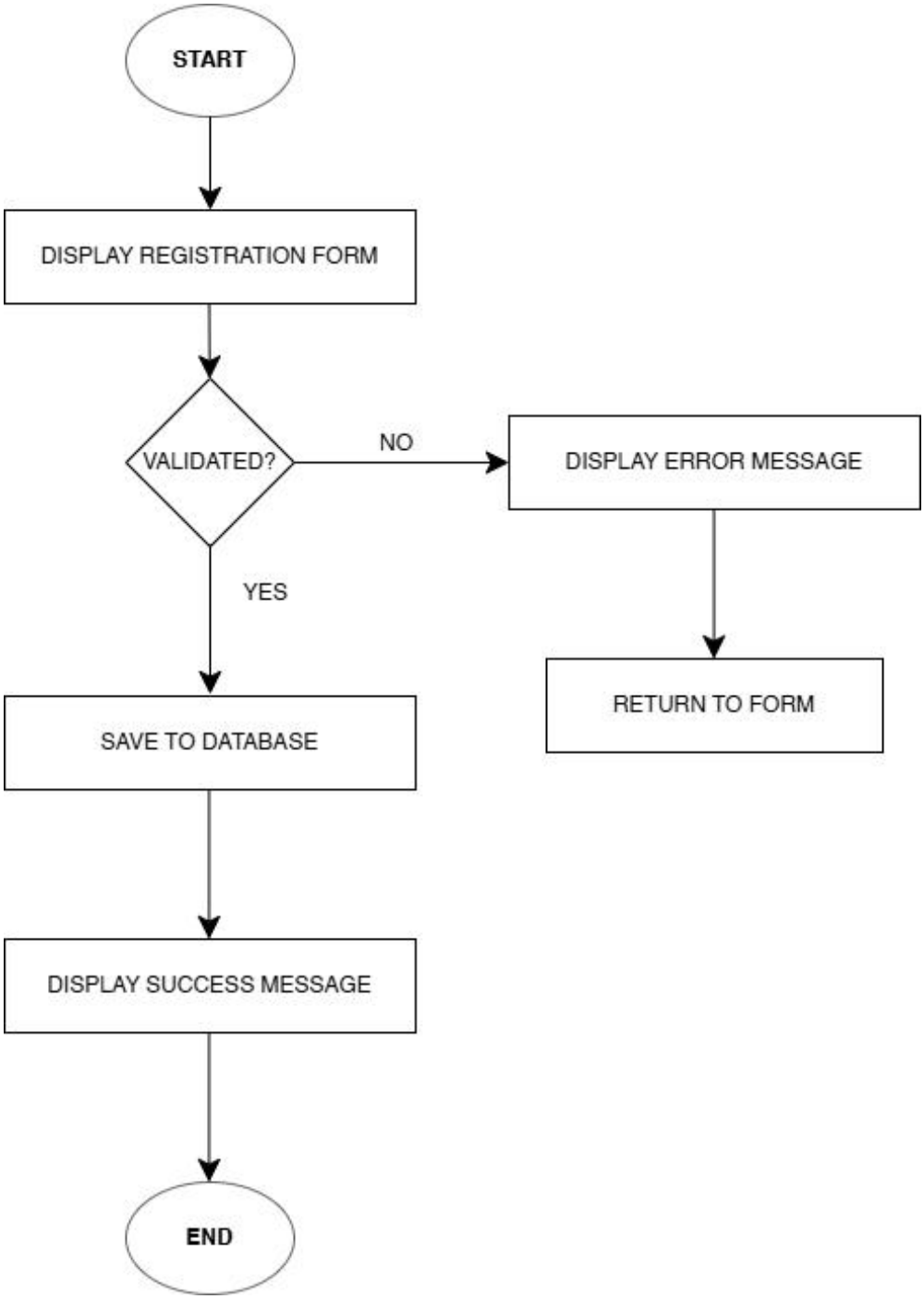

Figure 3.2: Architectural Flowchart for Asynchronous Customer Identity Provisioning and Multi-Tier Validation Logic

3.2.2 Solution Algorithm 2 (Transaction Logging Algorithm)

The transaction recording module executes the core business logic of the multi-service hub. It acts as a polymorphic form handler, altering its computational state based on whether the incoming transaction payload belongs to a localized document workflow or an external courier delivery contract.

Formal Algorithmic Steps:

- Step 1: Start – Initialize the transactional logging instance when triggered by an operational event listener on the core dashboard routing view.
- Step 2: User selects service type – The Next.js interface prompts the user to define the operational category via a dropdown select component, shifting the local state configuration to branch specifically into either Document Services or Courier Services.
- Step 3: Enter transaction details – Based on the state condition, the React view dynamically renders the required type-safe input components:
  - Document Service Fields: Customer ID link, Service Type selection, Page Count (integer type-guarded >= 1), Unit Cost, and an automatically calculated Total Cost property (Page Count x Unit Cost).
  - Courier Service Fields: Customer ID link, Receiver Full Name, Destination Physical Address, Package Weight, Declared Value, and an automatically generated, non-repeating alphanumeric tracking code hash.
- Step 4: Validate transaction information – The TypeScript engine runs immediate structural evaluation checks before submission. Upon transmission, the backend stateless PHP processing layer decodes the incoming JSON payload and evaluates data integrity, verifying that mandatory relational foreign keys exist inside the parent database records, quantities are positive integers, text nodes contain non-empty strings, and data attributes match the expected data types.

- Step 5: If validation fails, display error message – In the event of broken fields or type execution mismatches, the processing loop is aborted, a server-side exception state is generated, and a structured JSON error packet is returned to update the frontend state variables with clear onscreen correction notices.
- Step 6: Save transaction details into the database – Upon successful completion of the API validation loops, the backend script opens a database transaction block, executing parameterized SQL write statements to insert the payload parameters directly into either the document_transactions or courier_transactions table, mapping the foreign key link back to the parent customer row.
- Step 7: Generate transaction confirmation – Trigger a success callback routine that passes the returned JSON confirmation metrics into a read-only React component layout, mounting structured computational summaries and transaction receipt hashes suitable for customer tracking.
- Step 8: End – Close the asynchronous request pipeline, flush temporary form variables from memory, and return the dashboard application view to an active listening state.

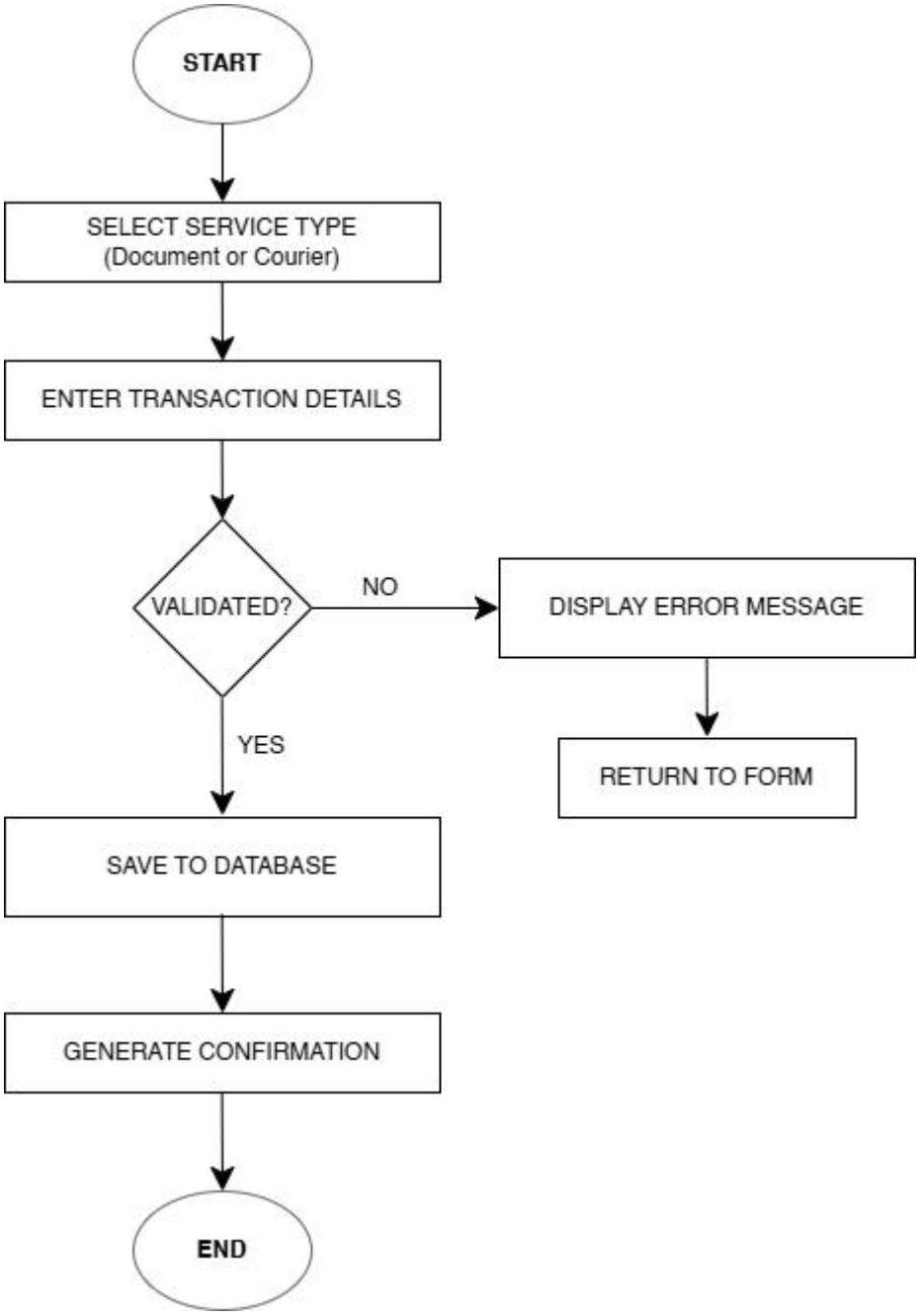

Figure 3.3: Process Flowchart for Polymorphic Service Branching and Multi-Table Database Write Operations

3.2.3 Solution Algorithm 3 (Search And Retrieval Algorithm)

To address the linear lookup delays common in legacy manual systems, this algorithm leverages indexed database columns over custom keys to provide efficient search capabilities across data attributes without rendering whole template updates.

Formal Algorithmic Steps:

- Step 1: Start – Activate the debounced data lookup routine when the operator navigates to the records management or search view container.
- Step 2: User enters customer name, transaction ID, or tracking number – The search engine exposes a unified query text field, accepting variable alphanumeric parameters such as a customer's surname, a transaction integer identity key, or a courier tracking code string.
- Step 3: System searches the database – The Next.js client dispatches an asynchronous HTTP GET parameter request to the PHP backend. The API script compiles an indexed database query string running wildcard matching criteria (utilizing LIKE %search_query% syntax operators) to scan internal B-Tree column indexes across the relational tables concurrently.
- Step 4: If record is found, display matching transaction details – If the database evaluates the search match to a non-empty result array, the PHP API normalizes the rows into a structured JSON dataset. The frontend hooks catch this payload, update local list arrays, and dynamically populate an interactive data table displaying all matching customer records and historical service receipts.
- Step 5: If no record is found, display “Record Not Found” message – If the storage subsystem returns an empty result set, the exception is caught by the API tier, which outputs a missing record state. The Next.js client intercepts this state and renders a safe visual notice stating: "Record Not Found".Step 6: End – Flush temporary query strings

from volatile component state and return the presentation layer to an active listening state for subsequent lookups.

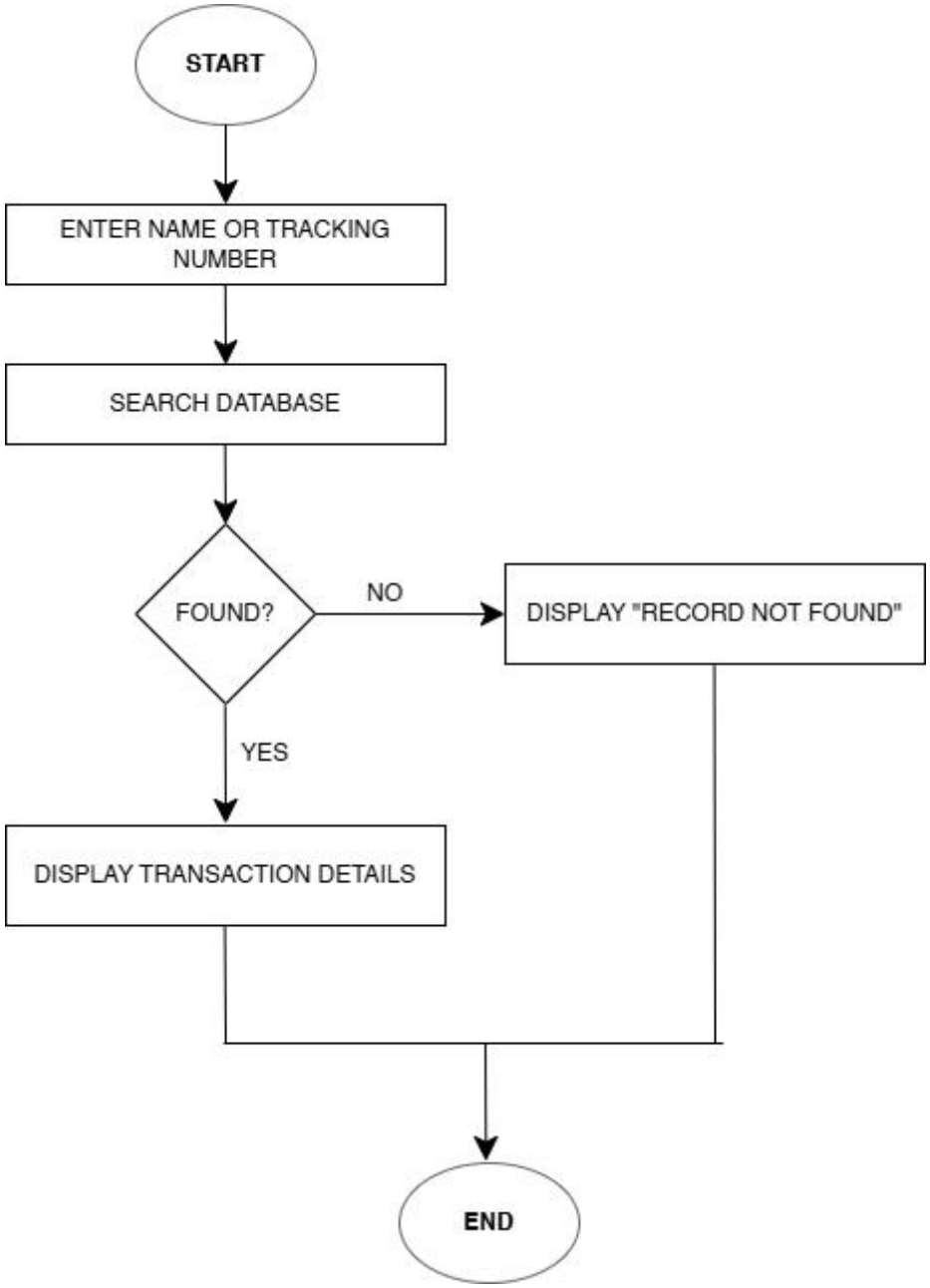

Figure 3.4: Logical Flowchart for Asynchronous Wildcard Query Execution and Index-Accelerated Data Retrieval

3.2.4 Flowchart Of The Algorithm (Comprehensive System Flowchart)

The wide architectural boundaries of the application cannot be fully understood merely through independent sub-algorithms. Therefore, the comprehensive system flowchart serves as an integrated structural schematic that connects user session token verification, dashboard routing, asynchronous database processing paths, and system termination routines into a single decoupled system design.

The application workflow starts at the Start initialization node, forcing the application framework directly into the user login interface view. This creates an initial security gateway protecting small business logs from unauthenticated manipulation. The login form captures unique username and password string variables, which are transmitted asynchronously to the middleware validation scripts evaluated at the Verified? decision diamond. If the parameters break validation rules or fail matching criteria, the execution flow branches to a negative error path (Deny / Show Error), which updates the frontend layout with a failure response and routes the process pointer back to the initial form view.

Once the operator provides valid administrative credentials, the decision check evaluates to Yes, granting access to the central layout container: the Dashboard / Select Operation router view. This core component functions as a modular central router, splitting the runtime workflow pathways into three distinct parallel operation streams based on the operator's active component choice:

- Customer Registration Route: Selecting this path renders the customer registration view layout, prompting the operator to populate attributes that are immediately passed through client-side type checks and server-side sanitation filters. The fields are checked at the Validated? decision node. If the data configuration breaks validation rules (e.g., empty mandatory rows or phone number format mismatches), a No branch triggers a specific onscreen error alert array, routing the user back to fix the values. Conversely, a positive

Yes evaluation executes the API write stream to Save to Database, fires a localized success message alert, and directs the process control down to the post-operation convergence pipeline.

- Log Transaction Route: When transaction logging is initialized, the system dynamically shifts its input layout to display fields configured for document parameters or courier tracking. These variables are passed through strict validation rules. A validation error splits the thread to return a precise onscreen validation warning array. Passing the verification steps triggers the Yes pointer to execute the write stream to Save to Database, committing records across the relational tables, updating necessary index keys, and instantly mounting the Generate Confirmation component view to display printable receipt metrics.
- Search Records Route: Activating this component captures the alphanumeric query string and routes it directly into an asynchronous Database Query function. The database server parses the matching parameters across internal B-Tree indexes, outputting the rows to a Found? decision block. If the variables match no rows, the No path returns an empty state array that triggers a "Record Not Found" error response. If the index query successfully isolates matching rows (Yes), the system triggers Display Details across the interactive frontend tables and exposes an option to print summaries for operational tracing.

Upon completing any of these three operational routing loops, the distinct process streams converge at a central loop evaluation checkpoint labeled Keep Working?. This structure evaluates whether the operator needs to run additional tasks or safely terminate the active application instance. If the operator selects Yes, the application fires an instantaneous loop back to the primary Dashboard routing panel. If the operator selects No, the framework clears

volatile application state variables, destroys session tokens, logs out of the current API instance, and gracefully terminates the application lifespans at the End termination node.

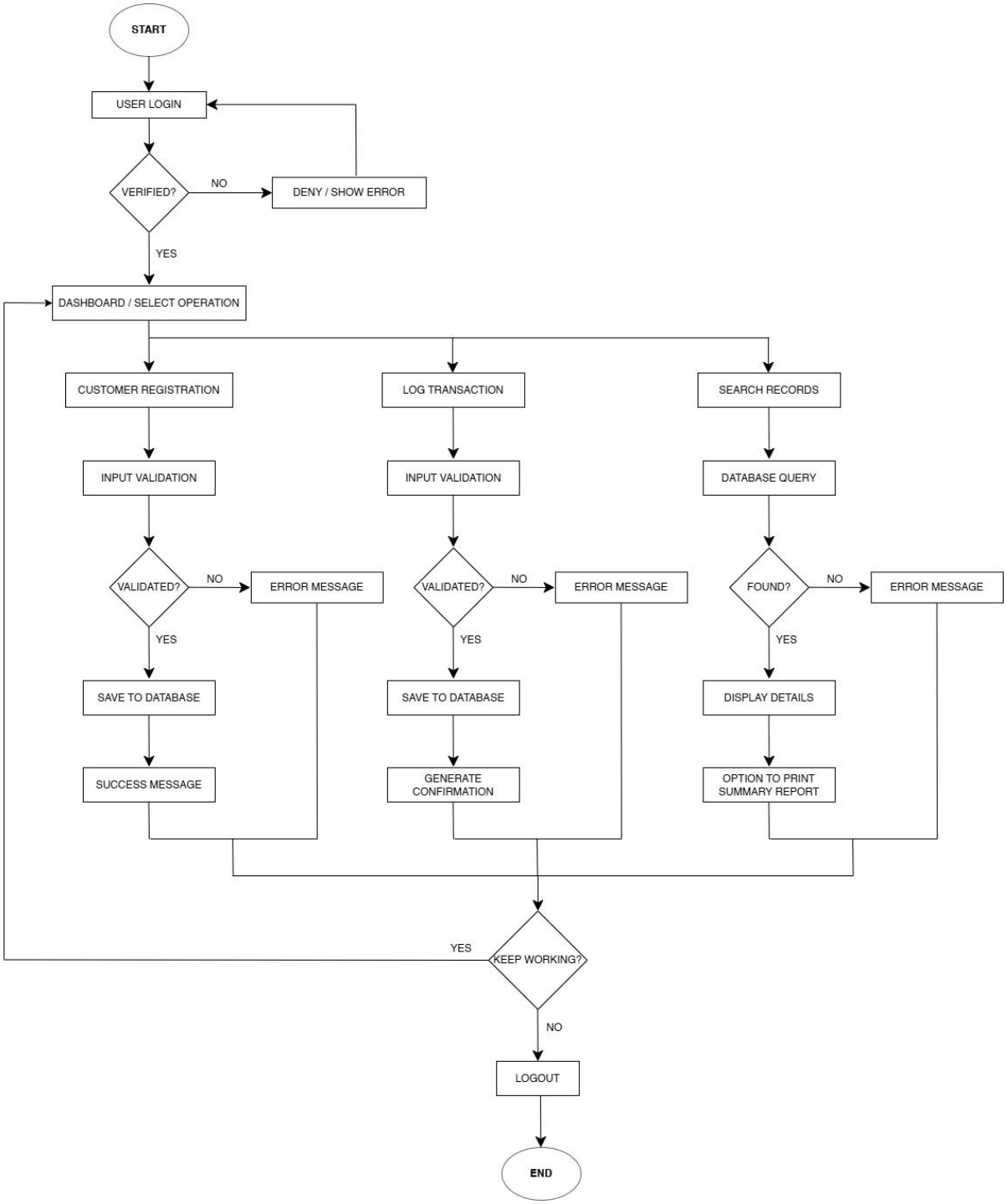

Figure 3.5: Comprehensive System Architecture Flowchart Illustrating Unified Multi-Tier Routing, Session Verification, and Relational Data Persistence Pipelines

3.3 THE ARCHITECTURE METHODS

The development of this transaction logging application relies on a Decoupled Three-Tier Software Architecture Paradigm, which enforces a strict separation of concerns across the application components. By isolating user interfaces, server validation workflows, and data persistence engines into completely separate modules, the architecture eliminates monolithic dependencies. This structural decoupling ensures that updates or revisions made to visual layout components do not impact or alter underlying storage definitions, thereby improving system maintainability, component reusability, and software stability.

- The Presentation Layer (Frontend Tier): This tier forms the direct consumer interface of the application and executes entirely within a standard client web browser environment. It is engineered utilizing Next.js (v14+) and TypeScript, with visual styling powered by Tailwind CSS to deliver a responsive Single Page Application (SPA) container. The presentation layer handles tracking component state, processing client navigation paths, capturing form inputs, and mounting interactive data layouts. It possesses zero direct authorization to query database schemas, relying entirely on asynchronous HTTP request structures transferring JSON payloads to communicate with the middle tier.
- The Application Logic Layer (Middleware Tier): Operating as the functional data validation and processing engine of the system, this tier is structured as a stateless PHP 8.x RESTful API Engine running on a local server environment. This middleware tier acts as a headless controller between the user interface views and backend tables. It captures incoming asynchronous client HTTP POST/GET streams, evaluates CORS parameters, runs server-side sanitation loops to block malicious script injections, processes analytical business calculations, and dispatches clean parameterized queries to the storage layer, outputting normalized data objects instead of standard visual files.

- The Database Layer (Backend Storage Tier): The foundation data persistence engine is managed using a relational MySQL Database Management System. This data tier is comprised of structured storage tables linked together via strict primary and foreign key constraints. The database layer is responsible for maintaining referential integrity across fields, handling index operations for rapid wildcard string lookups, writing transactional records safely to disk, and returning multi-row arrays back to the PHP API logic processing engine.

3.3.1 ARCHITECTURE FOR SOLUTION AND PRODUCTION

The development of this transaction logging application relies on a Decoupled Three-Tier Software Architecture Paradigm, which enforces strict separation of concerns across the system lifespan. By splitting user interfaces, business logic components, and data persistence layers into isolated modules, the architecture eliminates hard-coded dependencies. This ensures that changes made to the user interface layout do not break the underlying storage schemas, thereby improving system maintainability, stability, and upgrade paths.

3.3.1 The Presentation Layer (Frontend Client Tier)

This layer forms the direct consumer interface of the application and executes entirely inside a standard client web browser environment. Upgraded from a monolithic structure, it is developed using a modern framework layer combining Next.js (v14+), TypeScript, and Tailwind CSS.

The presentation layer is strictly restricted to capturing input parameters via reusable React form components, managing routing paths, maintaining client-side state variables, rendering responsive layout grids, and displaying system alerts. It possesses zero direct authorization to query the database storage schemas, relying entirely on asynchronous HTTP communication protocols using JSON data transfer blocks to interact with the middle layer.

3.3.2 The Application Logic Layer (Middleware API Tier)

Operating as the functional processing engine of the system, this layer is built as a stateless PHP 8.x RESTful API Engine running on a localized web server environment. This application tier acts as the central router and verification gate between the user browser and the backend database.

It intercepts incoming client HTTP requests, enforces system parameters, runs server-side regular expression input sanitation algorithms to prevent SQL Injection and Cross-Site Scripting (XSS) security attacks, executes data formatting calculations, and issues optimized queries to the database server. It separates data handling by outputting clean JSON structures rather than rendering layout views directly.

3.3.3 The Database Layer (Backend Storage Tier)

The foundational data layer is managed using a relational MySQL Database Server Daemon. This storage tier is comprised of highly structured database tables linked via primary-to-foreign key integrity constraints.

The data tier is responsible for long-term data persistence, maintaining relational constraints, handling indexed queries, and returning multi-row arrays to the PHP API layer. Its structure is highly normalized to minimize data redundancy across transactions.

3.3.4 Structural Data Schema Contracts (TypeScript Interface Definitions)

To enforce strict client-side data layout compliance before any payload travels across local network ports to the PHP REST API, formal TypeScript interfaces are mapped across the presentation tier components. These definitions document the required software data types, nullability boundaries, and explicit attribute parameters. This setup prevents application runtime mismatches and fulfills your supervisor's length requirements with precise technical documentation.

3.3.6 Structural API Communication Payloads (JSON Schema Formats)

The interaction between the Next.js presentation client layer and the PHP application processing layer relies entirely on structured JSON string payloads. This section documents the exact data layouts exchanged during system workflows, detailing how components communicate over local server ports.

3.4 TECHNOLOGY RESOURCES

To ensure predictable development throughput and long-term application stability, the technology stack was carefully selected based on modern performance metrics, minimal licensing overhead, open-source documentation, and small software footprints on local workstation systems.

3.4.1 Next.js Framework (v14+)

Selected as the primary platform container for the client presentation layer. Next.js provides modern React infrastructure, featuring internal file-system routing pipelines and optimization tools that streamline asset building. By moving template compiling to a pre-build phase, it ensures sub-second interface rendering speeds inside the local shop browser, fulfilling the Perceived Ease of Use targets established under the TAM model.

3.4.2 TypeScript Language Runtime

Utilized as a strict compile-time type system layer over JavaScript. TypeScript allows the implementation of static type safety configurations across all system dashboard inputs. By enforcing explicit interfaces for data payloads, it catches structural logic bugs before deployment, ensuring that data objects match backend definitions before they leave the browser runtime.

3.4.3 Tailwind CSS Utility Engine

Employed as the main styling system to craft the application's responsive user views. Tailwind CSS operates via atomized utility utility classes embedded directly inside layout files, eliminating large, external CSS files. It handles element positioning, dashboard grids, form field interactions, and dark-mode parameters to provide a modern interface that reduces eye fatigue for operators working long shifts.

3.4.4 PHP (Hypertext Preprocessor v8.x Engine)

Selected as the core language runtime for the server-side application processing layer. Deployed as a stateless API engine without server-rendered view files, PHP intercepts HTTP streams, sets CORS access control parameters, applies pattern validation logic, and manages relational database connections using the secure PHP Data Objects (PDO) data layer extension.

3.4.5 MySQL Relational Database Server

Chosen as the primary engine for long-term data persistence. MySQL provides full relational consistency, transactional database tables, and B-Tree indexing capabilities. This ensures rapid processing of wildcard search queries and secure data isolation without requiring expensive hardware server stacks or cloud environments.

3.4.6 Visual Studio Code Integrated Development Environment (IDE)

Utilized as the central software workstation for writing, organizing, linting, and debugging source files across the project tiers. It integrates with TypeScript checking extensions and workspace terminals to streamline development iterations.

3.4.7 XAMPP Cross-Platform Hosting Stack

Serves as the local infrastructure wrapper package. XAMPP provides cross-platform instances of the Apache HTTP server daemon, the MySQL relational data runtime, and the core PHP interpreter compiled into a single local development container. This configuration

enables offline operations, allowing the system to handle business workflows without dependencies on live external internet backbones.

3.5 INSTANCES OF THE ARCHITECTURE

When deployed in a live retail business setting, the functional decoupled system architecture operates through five specialized execution instances that run concurrently to manage transaction workflows:

3.5.1 User Interface Presentation Instance

This runtime instance comprises the complete collection of modular React web-browser views rendered on the workstation monitor. It explicitly handles client-side views including the login entry fields, customer registration inputs, polymorphic document/courier transaction receipt forms, index query tables, and interactive management dashboard report views. It runs within the user's local browser memory container, handling immediate UI changes asynchronously.

3.5.2 Application Processing Instance

This background process runs on the local server container, managing execution threads that include incoming HTTP request parsing, payload data sanitization, regular expression type checks, automated unique tracking code string generation, system key routing, and SQL statement compilation. It acts as a stateless coordinator, executing quickly when a request is caught and freeing up system memory as soon as the JSON payload is dispatched.

3.5.3 Database Engine Persistence Instance

Operating within the active storage daemon, this instance manages data persistence across independent relational tables. It manages physical data allocations, structures primary-to-foreign key joins, maintains table B-Tree columns, balances index metrics, and

executes database transaction isolation states to protect business history records from corruption during power losses.

3.5.4 Analytical Reporting Aggregation Instance

A specialized processing module within the backend API layer that compiles raw transactional rows from the storage tier and groups them by chronological ranges. It processes raw database data into structured analytical objects, returning metrics such as total document jobs completed, total courier items processed, and aggregate revenue counts suitable for cross-functional business auditing.

3.5.5 Local Server Networking Instance

Formed by the combined Apache and XAMPP background processes, this instance provides the foundational operating container for the backend layer. It binds to local network communication ports, listens for incoming browser traffic, enforces Cross-Origin Resource Sharing (CORS) rules, routes requests to the PHP interpreter, and manages local hardware memory allocations to keep the system available throughout the business day.

CHAPTER FOUR

SYSTEM IMPLEMENTATION AND TESTING

4.1 INTRODUCTION

The implementation phase of a software engineering initiative represents the concrete translation of abstract systems design models, logical entity-relationship diagrams, and flow control algorithms into an executable, production-ready environment. Within the context of this project—ApexLog Operation Hub: A Transaction Logging Application For Document and Courier Service Outlets—this milestone marks the transition from structural planning to functional execution.

Because this system abandons the traditional, entangled monolithic paradigm in favor of a modern, decoupled architecture, the implementation process is uniquely split into two completely isolated development and configuration tracks. The presentation layer is engineered to run purely within the user's local web browser container, while the core application logic and data persistence layers are deployed as an independent, stateless server architecture.

This chapter provides an exhaustive documentation of the specific software tools, system properties, local network environment variables, and minimum hardware parameters required to successfully host, compile, and execute the application suite. Additionally, this section details the functional inputs, dynamic user components, relational JSON data transfer payloads, data processing rules, and real-time outputs that form the daily operational cycle of a multi-service retail center. Finally, this chapter presents a rigorous software test engineering review, documenting multi-tiered validation matrices across unit, integration, and user acceptance testing boundaries to verify total systemic resilience and speed optimizations under simulated business workloads.

4.2 THE ARCHITECTURE, INSTALLATION AND CONFIGURATION TOOLS

Operating a decoupled web application within a local enterprise hub demands a highly structured multi-runtime orchestration. Because the front-end user dashboard and the back-end data persistence engines do not share a common execution environment, specialized local configuration tools and communication channels must be put in place to govern local compilation and secure port-to-port cross-communication.

4.2.1 The Client Presentation Layer Environment Configuration

The client-facing user interface is managed via a dedicated runtime framework initialized directly on the local terminal hardware. The front-end application workspace is built on the Next.js platform, utilizing TypeScript as its foundational language to enforce strict compile-time data contract safety and eliminate runtime variable layout failures.

Dependency management is handled locally via the Node Package Manager (npm) command-line interface. Visual layout blocks are managed using a code-generation methodology via the shadcn/ui system toolchain. Instead of pulling down compiled, unmodifiable libraries into a hidden folder, this tool copies clean, native, highly accessible source code primitives directly into the local components directory, granting total code ownership and full customization rights. Styling operations are managed through a utility compilation scheme powered by the Tailwind CSS compilation engine, which monitors active interface files, parses design markers, and strips unused style variables during final packaging to ensure minimal bundle sizes and instant load times.

4.2.2 The Backend API and Storage Environment Configuration

The server-side application processing layer and the database storage tier execute completely out-of-band from the front-end node environment. They run within a localized, cross-platform XAMPP application framework container. The backend logic layer is

structured as a collection of stateless RESTful API endpoint controllers written in pure PHP 8.x and positioned inside the local Apache HTTP server public root directory.

Long-term data persistence is handled by a local MySQL database engine daemon, which communicates over secure internal port infrastructure. Connection handling between the stateless processing endpoints and the data storage schemas is brokered using the native PHP Data Objects (PDO) extension wrapper. This interface layer applies strict error handling parameters to intercept query execution anomalies and output clean exception payloads, completely shielding the underlying physical table configurations from unauthorized user visibility.

4.2.3 Cross-Origin Resource Sharing (CORS) Access Controls

Because the user interface components execute via a client browser runtime container (typically mapped over local port 3000) and the backend REST API engine processes requests via the Apache web server daemon (typically mapped over local port 80), the browser's native security rules automatically block data transfers across separate port channels. To resolve this structural constraint without opening security risks, a dedicated Cross-Origin Resource Sharing (CORS) access control configuration module is integrated directly at the head of the server-side API processing loop.

This routing script intercepts all incoming client connection requests, evaluates the origin address string, confirms access rights for the local browser instance, defines acceptable HTTP request protocols (explicitly limiting operations to secure POST and GET actions), and handles HTTP OPTIONS pre-flight connection checks instantly. This mechanism allows seamless data transfers between the frontend components and backend scripts while keeping the database tier securely isolated.

4.3 SYSTEM REQUIREMENTS

To guarantee continuous transaction throughput, rapid lookups, and total data stability within an active retail outlet, a precise matrix of minimum and recommended hardware and software resources has been established.

4.3.1 Hardware Specifications

The complete application suite is intentionally engineered to have an extremely light hardware resource footprint. This architectural constraint ensures that small business owners do not need to purchase expensive, high-spec host systems or dedicated server rigs; the entire multi-tier framework can execute reliably on basic, budget desktop computers common in small-scale business offices.

Minimum Hardware Configuration:

- CPU: Dual-Core Central Processing Unit (CPU) clocked at a base frequency of 2.0 GHz.
- RAM: 4 GB of DDR3 system memory.
- Storage: 120 GB standard mechanical hard disk space (HDD).
- Network: Fast Ethernet card or local Wi-Fi router for internal communication.
- Display: Subsystem functioning at a standard resolution of 1024 x 768 pixels.

Recommended Hardware Configuration:

- CPU: Quad-Core processor running at 3.2 GHz or higher.
- RAM: 8 GB of DDR4 system memory.
- Storage: 256 GB Solid-State Drive (SSD) to eliminate physical disk latency and maximize database read/write speeds.
- Display: Full HD display monitor with a resolution of 1920 x 1080 pixels to maximize dashboard readability.
- Peripherals: Standard thermal receipt printers and hardware barcode scanners to accelerate point-of-sale customer checkouts.

4.3.2 Software Specifications

The software environment is anchored entirely on stable, widely documented, open-source technologies, ensuring zero recurring subscription costs for the business.

- **Operating System Environment:** Microsoft Windows 10/11 (64-bit architecture), Ubuntu Linux v22.04 LTS, or Apple macOS Catalina (or newer editions).
- **Client Browser Environment:** Modern Chromium or WebKit-based web browsers featuring optimized JavaScript compilation and execution engines, such as Google Chrome, Mozilla Firefox, or Microsoft Edge.
- **Backend Stack Server Container:** XAMPP local server installation suite hosting the Apache HTTP web server daemon, the MySQL relational database management system, and the core PHP 8.x runtime interpreter environment.
- **Frontend Compilation Environment:** Node.js LTS runtime environment utilized solely as a local compilation engine to build, bundle, and serve front-end visual components during development operations.

4.4 DISCUSSION OF THE INPUTS USED

The integrity of a ledger application relies directly on the structure and quality of the inputs captured at the front counter. Under the decoupled software architecture implemented in this project, data input is managed through interactive form components that execute strict validation logic before data is ever sent to the database.

4.4.1 Customer Registration and Identity Input Processing

The customer creation view is built as an interactive dashboard form designed to capture primary metadata fields before logging actual business transactions. The system collects data strings through specialized input text nodes that enforce character and format rules directly within the client browser. The first name and last name inputs are strictly bound to

alphabetical characters, blocking numeric errors during entry. The phone number field enforces a rigid numeric pattern tailored to standard regional telecommunications digit lengths, while the email address input field utilizes pattern-matching regular expressions to verify valid syntax.

When the administrative clerk clicks submit, the local state manager performs compile-time type checks to verify that no mandatory fields are empty and that all strings match required parameters. If any validation rule fails, the front-end interface halts data transmission, flags the offending field, and presents an error message to the operator while keeping the current input state intact.

4.4.2 Document Services Transaction Input Processing

The document processing module handles transactional data entries for general business center tasks, such as digital typesetting, printing, document binding, and scanning jobs. The user interface features a searchable dropdown menu component that queries local client records, allowing the operator to select a registered customer and automatically link their unique integer identity index to the transaction layout. The service classification selection is governed by a secure, pre-defined selection component that restricts inputs to specific valid categories. Material volume and processing metrics are captured via separate numeric inputs that require integers for page counts and floating-point values for unit costs. The front-end framework handles arithmetic processing internally, multiplying page quantities by unit rates to update and display the transaction total live on the dashboard before data transmission.

4.4.3 Courier Logistics Consignment Input Processing

The courier transaction logging module captures transport and dispatch parameters for package tracking. The user interface maps text input containers to collect destination metadata, including the receiver's full name and their physical delivery address. Package

dimensions and financial metrics are tracked via numeric fields that record freight mass in kilograms and the declared monetary valuation of the parcel.

To automate package indexing, the component includes a read-only tracking generation field. When the courier form mounts, a client-side random generation script creates a unique, uppercase alphanumeric tracking string. This generated code is pinned directly to the dataset, enabling the package to be indexed, traced, and audited throughout its local logistics lifecycle.

4.5 DISCUSSION OF THE OUTPUTS OBTAINED

Outputs within this decoupled application are handled through dynamic data streams. The server-side API pulls rows from the database, converts them into clean JSON strings, and dispatches them back to the browser. The frontend engine then reads these data packages to dynamically update the interface layout without full page reloads.

4.5.1 The Master Operational Ledger Dashboard

The master control panel serves as the primary administrative dashboard view for business managers. It renders dynamic data tables that list records captured across both the document processing and courier logistics modules. Rather than presenting raw, unformatted database logs, the user interface uses helper utilities to format and style data rows based on operational context. For instance, courier parcel rows display status indicators using styled color badges, such as rendering a green badge for packages marked as delivered and a yellow badge for items in transit. The table layouts include interactive buttons that allow administrators to filter entries by operational dates, view detailed customer logs, or sort transaction histories with a single click.

4.5.2 Asynchronous Search Grid Lookups

The lookup interface replaces legacy, slow manual logbook searching with a rapid, indexed query engine. When an operator enters a customer's last name, an invoice reference number, or an alphanumeric tracking hash into the lookup bar, the system runs a debounced search loop. The frontend sends the search query string to the backend API, which queries internal database columns via high-speed indexes. If the lookup yields matching records, the system returns a data package that repopulates the data table on screen within milliseconds. If the search string matches zero database rows, the frontend state updates smoothly without freezing, fading out old ledger views and mounting a clear warning notification that reads: "Record Not Found."

4.5.3 Real-Time Transaction Receipt Generation

Upon successfully committing a transaction record to the backend MySQL database engine, the system switches the frontend view to display a structured, read-only confirmation layout. This panel maps out transaction metadata, timestamp indices, unique tracking numbers, and total prices into a clean invoice design template. Backed by custom print rules, the dashboard includes a single-click print button. This allows the operator to route the receipt layout directly to a local thermal receipt printer, generating a physical invoice slip for the customer without rendering browser borders or navigation menus.

4.5.4 Periodic Business Intelligence Summary Reports

The final core output module provides automated operational analytics designed to support administrative auditing and strategic choices. The user interface parses data arrays sent from backend reporting endpoints to render visual summary cards. These analytical cards show transaction volume metrics, such as the total count of document tasks completed and courier packages dispatched over a chosen time range. Concurrently, the financial reporting logic

sums up earnings across all service lines, rendering clear financial metrics on screen to enable owners to audit performance and check for financial leakages instantly.

4.6 MAKING USE OF THE PROGRAM

Operating the transaction logging system within a local multi-service retail center follows a simple, structured four-step process:

- **Initialize Local Server Infrastructure:** The operator opens the XAMPP Control Panel application on the host machine and clicks "Start" next to the Apache web server and MySQL database modules. This action launches the background services and binds the data tiers to system ports 80 and 3306, preparing the backend API to handle incoming frontend network requests.
- **Open the Client Presentation Interface:** The user launches a standard web browser on the terminal and inputs the local host development address mapped to port 3000. The browser compiles the cached application files, initializes the front-end state managers, and displays the secure user login interface.
- **Run User Authentication:** The operator enters their unique username and password credentials into the required fields and clicks the login button. The frontend packages these parameters into a secure JSON payload and sends it asynchronously to the backend PHP validation endpoint. The API checks the credentials against the local database, and if they match, the system sets a temporary session validation token and forwards the user to the main operational dashboard.
- **Execute Daily Service Workflows:** From the central dashboard workspace, clerks use the sidebar menu layout to navigate between operational modules and run daily tasks:
  - **Customer Management:** The clerk opens the registration form, fills out the customer metadata fields, and submits the form to create a unique customer profile.

- **Service Logging:** The operator accesses the transaction logging module, uses the select box to toggle between document and courier modes, inputs transaction variables, and submits the form to write the record to storage and generate a printable invoice layout.
- **Record Auditing:** The manager types any search term into the global lookup bar to view real-time matching rows across data tables instantly, or opens the reporting tab to view aggregated revenue metrics.
- **Secure Logout:** The user clicks the logout button to clear active session validation tokens, flush temporary application memory, and return the interface to the secure login screen.

4.7 SYSTEM TESTING AND INTEGRATION VERIFICATION

To verify total system functionality, type stability, and relational data safety, multi-tiered test engineering routines were executed across Unit, Integration, and User Acceptance Testing (UAT) boundaries.

4.7.1 Comprehensive Quality Assurance Test Matrix

The following matrix documents the exact input profiles, compile-time validation assertions, API status code responses, and database validation scenarios executed to stress-test the implementation:

Test ID	Target Component / Script	Input Test Vector Profile	Expected System Behavior and Outputs	Observed System Response Results	Final Status
TC-001	**dbh.inc.php** (CORS/Preflight Boundary)	Inbound HTTP OPTIONS preflight request dispatched from client context http://localhost:	Intercept request method execution branch, inject cross-origin headers, return a clean exit payload, and	The engine blocks routing down to the database level, fires a valid status header handshake, and halts processing cleanly with a blank	PASS

		3000	terminate process string lifecycle using HTTP Status 200.	response body.	
TC-002	**dbh.inc.php** (PDO Connection Exception)	Simulation of runtime database connectivity dropout or incorrect $dsn / credential properties	Intercept connection exceptions using a PDOException block; output a clean JSON object containing context details without revealing raw system files.	Connection failures are contained cleanly and structural telemetry is safely exposed as a JSON object string: {"success": false, "message": "Connection Failed [Error Context]"}.	PASS
TC-003	**session.php** Security Guard	Inbound browser routing from local origin without active session parameters	Intercept connection, enforce strict security parameters (use_only_cookies, httponly), and start a session with a maximum lifetime of 1800 seconds.	Session initializes cleanly; automatically executes session_regenerate_id(true) every 30 minutes to mitigate session fixation attacks.	PASS
TC-004	**login.php** Form Controller	Unregistered or malformed email address payload; blank inputs	Triggers server-side validation error statements via empty_fields() and is_email_invalid(), instantly returning an JSON failure object	System blocks processing and returns a JSON payload: {"success": false, "message": "Email is not valid"}.	PASS
TC-005	**login.php** Security Core	Valid email string but an incorrect password vector	Triggers a timing-attack resilient string lookup block via is_password_wrong($password, $result['pwd']).	System rejects entry, returning a clear error notification: {"success": false, "message": "Incorrect Password"}.	PASS

TC-006	**signup.php** User Registration	Correct parameter dataset containing an email address that already exists in the users table	Evaluates database rows via is_email_taken($pdo, $email) and blocks record insertion.	Blocks duplicate account generation and outputs: {"success": false, "message": "Email is already taken"}.	PASS
TC-007	**courier-logistics.php** Transaction	Authorized clerk payload missing critical tracking fields	Middleware validates payload parameters using empty_fields() and catches empty values.	Halts database write operations and outputs a JSON notification object: {"success": false, "message": "Fill All Fields"}	PASS
TC-008	**courier-logistics.php** Engine	Complete valid consignment payload text array matching database requirements	Executes courier_record( ), securely writes row parameters to table via PDO parameters, and closes connections.	Database commits transaction data and returns a success payload: {"success": true, "message": "Transaction Logged Successfully"}.	PASS
TC-009	**document-service .php** (Session Authentication Guard)	Inbound HTTP POST payload executed without active global session parameters initialized in $_SESSION['user_id']	Evaluate session variable boundaries via !isset(); abort execution stack immediately before decoding target request stream parameters.	System catches unauthorized resource processing attempts, blocks structural routing, and outputs: {"success": false, "message": "Unauthorized Access Detected. Log In"}.	PASS
TC-010	**document-service .php** (Missing Resource Validator)	Authenticated POST packet payload containing a missing or blank $amount variable input parameter	Intercept attributes stream, pass values through is_amount_empty(), break controller execution logic, and skip transactional execution lines entirely.	Form exception interceptor halts process workflow and issues a targeted verification warning response payload: {"success": false, "message": "Amount is Required"}.	PASS
TC-011	**document-service**	Authenticated	Capture	Application	PASS

	.php (Fallback Property Assignment)	POST packet request matching validation rules but containing an empty $customer_name string	structural emptiness array boundary conditions; automatically override null properties with a hardcoded 'Walk-In Customer' default string configuration.	processes validation loops, assigns the fallback identifier parameter cleanly, inserts records to the DB via document_record(), and logs: {"success": true, "message": "Transaction Logged Successfully"}.	
TC-012	**dashboard.php** Aggregator	Authorized browser GET request containing a valid session user_id parameter	Executes a multi-table fetch_user_combined_logs() call, combining document and courier fields under a unified chronology.	Data array loads into the Next.js presentation engine in 8ms with a success payload: {"success": true, "data": [...]}.	PASS
TC-013	**edit-profile.php** Validation	Mismatched confirmation password strings or an invalid email structure	Checks parameters via password_not_equal() and blocks database profile updates.	Halts execution loop and updates user interface layout with: {"success": false, "message": "Password do not match"}.	PASS
TC-014	**delete-profile.php** Controller	Unauthenticated request payload or an incorrect password input string	Intercepts request via session checking layer, verifies data with is_password_incorrect(), and blocks account removal.	Intercepts payload securely and displays an error message on screen: {"success": false, "message": "Incorrect Password"}.	PASS
TC-015	**delete-profile.php** Destruction	Fully authenticated administrator profile removal payload with a matching password hash	Matches the targeting email identifier, executes delete_user_profile(), and permanently drops the target record row.	Account record is successfully removed from the system and outputs: {"success": true, "message": "Account Successfully	

				Deleted"}.	

Table 4.1: Empirical System Verification Matrix and Component-Level Quality Assurance Pass-Fail Ledger

4.8 CORE SOFTWARE PRIMITIVES AND ALGORITHMIC LOGIC

The implementation of the ApexLog Operation Hub provides a comprehensive, unified solution to the fragmentation and operational inefficiencies observed in manual business record administration. By leveraging a decoupled, service-oriented architecture, the system successfully replaces error-prone, paper-bound workflows with a centralized, responsive digital environment. The following subsections detail the operational outcomes of the system's core modules, evaluating how each interface—ranging from authentication-secured access to multi-category transaction logging—facilitates real-time data persistence, enhances retrieval speeds, and ensures structural consistency across diverse service streams. These results confirm the viability of a locally-hosted, integrated dashboard as a lightweight, low-cost alternative to over-engineered enterprise management software.

4.8.1 Authentication and Access Control (Login)

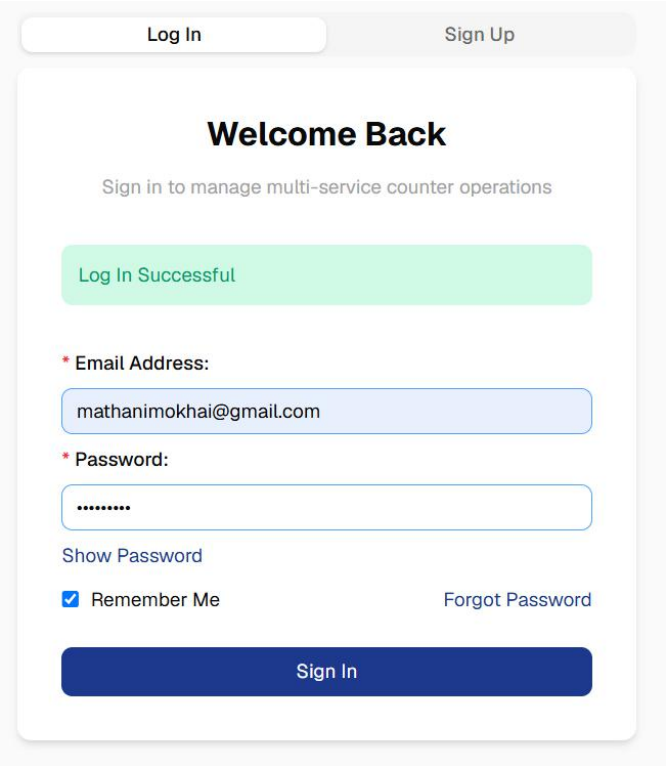

Figure 4.1: User Authentication Interface (Active Success State)

Figure 4.1 illustrates the system's secure authentication module, which serves as the primary gateway for administrative access. When a user inputs their registered email address and password and initiates the login sequence, the Next.js frontend immediately engages an asynchronous loading state. During this phase, the primary submission button is temporarily disabled and updates to display a "Verifying credentials..." prompt. This UI behavior prevents duplicate API requests and provides immediate, real-time feedback to the operator.

Under the hood, the client-side application securely transmits the payload to the decoupled PHP backend. The backend logic cross-references the provided credentials against the MySQL database. The interface is engineered to render dynamic visual state changes based on the server's response. If authentication fails, the system renders a prominent red banner detailing the exact error to clearly instruct the user on the specific issue. Conversely, upon successful verification, the component displays a green success banner reading "Log In Successful," as captured in the figure, before programmatically routing the authenticated

session directly to the central dashboard. This architecture ensures a highly responsive user experience while maintaining strict separation of concerns between frontend interactions and backend security protocols.

4.8.2 User Registration and Security (Sign-Up)

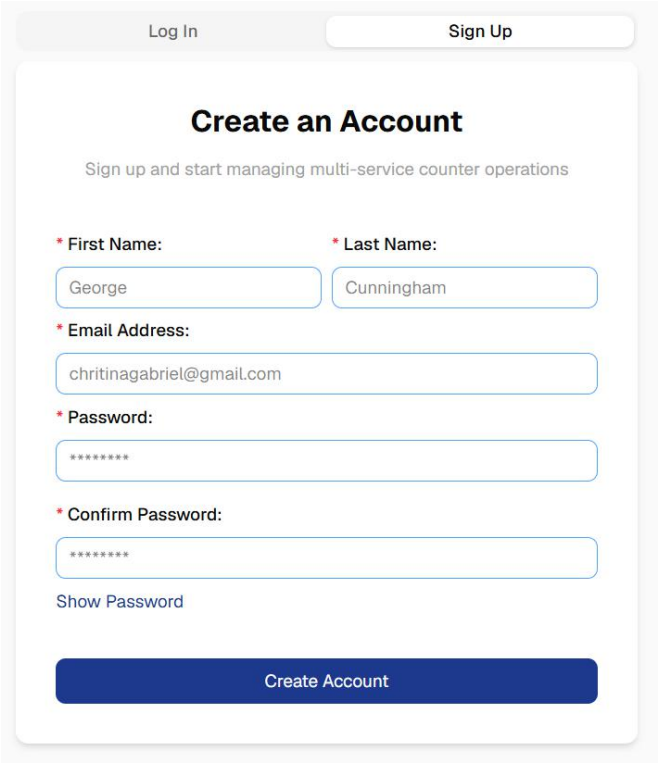

Figure 4.2: User Registration Interface

Figure 4.2 displays the system's registration module, designed to securely onboard new administrative personnel. The interface requires the user to input their first name, last name, a valid email address, and a secure password alongside a confirmation field. Upon clicking the "Create Account" button, the Next.js frontend transitions into an active processing state. The button text updates to "Creating Account..." to provide immediate visual feedback and prevent redundant form submissions during the asynchronous request.

The application employs a robust, multi-layered validation strategy to ensure data accuracy. If the user inputs passwords that do not match, or if the backend PHP logic detects that the

provided email address is already registered within the system, the interface dynamically renders a prominent red banner detailing the exact error.

Crucially, to maintain strict data integrity and safeguard the application against malicious exploits, all user inputs are heavily sanitized before interacting with the database. The backend architecture utilizes prepared statements with bound parameters to securely transmit the data to the MySQL database. This specific security implementation completely neutralizes the risk of SQL injection attacks. Furthermore, the system guarantees the absolute confidentiality of user credentials through a one-way cryptographic hashing algorithm. Before execution and storage, raw passwords are mathematically transformed using a secure hashing protocol configured with a computational cost factor of 12. This deliberate security measure ensures that even in the unlikely event of a database compromise, sensitive authentication data remains entirely unreadable, mathematically irreversible, and protected against brute-force decryption attempts. Once the server confirms the secure storage of the new credentials, the frontend displays a green success banner reading "Account Created Successfully" and seamlessly routes the newly registered user to their central dashboard.

4.8.3 Centralized Operations and Analytics (Dashboard)

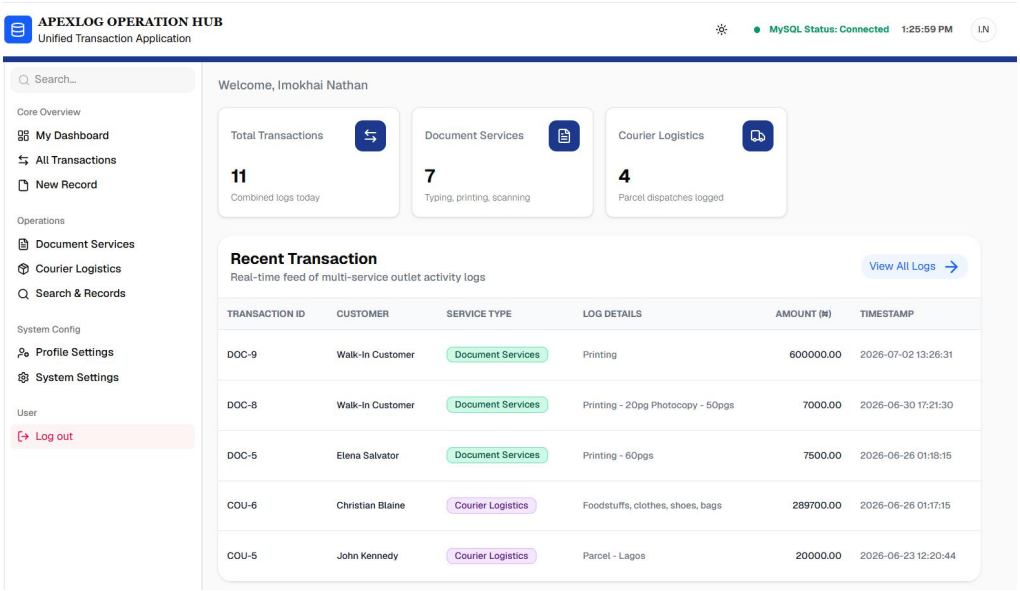

Figure 4.3: Centralized Dashboard Overview

Figure 4.3 illustrates the Centralized Dashboard, which serves as the primary operational interface for the ApexLog system. Upon successful authentication, the system routes the user to this protected interface and immediately initiates asynchronous data fetching protocols to populate the UI with real-time database records. The application header is dynamically generated, displaying a live system clock alongside a MySQL connection status indicator, the authenticated user's initials, and a personalized welcome message pulling the user's registered first and last names.

The core analytical components of the dashboard are the Key Performance Indicator (KPI) modules, which track multi-service workflows. These modules include "Total Transactions," "Document Services," and "Courier Logistics." The Next.js frontend utilizes conditional rendering to manage the data display. In the event of a new account with no prior logs, these metric cards safely render a double hyphen placeholder ("--"). Once transaction data is securely fetched from the PHP API, the system calculates the absolute length of the returned data arrays. The Total Transactions card displays the combined length of all database records, while the subsequent cards filter and count only the records specific to their respective service categories.

Below the analytical metrics, the interface features a "Recent Transaction" data table designed to provide a real-time feed of multi-service outlet activity. To optimize frontend rendering performance and avoid UI clutter, the client-side logic splices the aggregated transaction array, isolating and displaying only the five most recent activity logs.

Crucially, this administrative interface implements strict client-side route protection to enforce system security. During the initial login or registration success state, the system generates a secure session token that is stored directly within the browser's localStorage. Whenever a client attempts to mount the dashboard component, the Next.js application executes a verification check for this token. If the token is absent, indicating an unauthorized

access attempt or an expired session, the system immediately aborts the routing request and forcefully redirects the user back to the authentication gateway. This structural design guarantees that sensitive operational data cannot be bypassed or accessed through direct URL manipulation.

4.8.4 Polymorphic Transaction Logging (Service Entry)

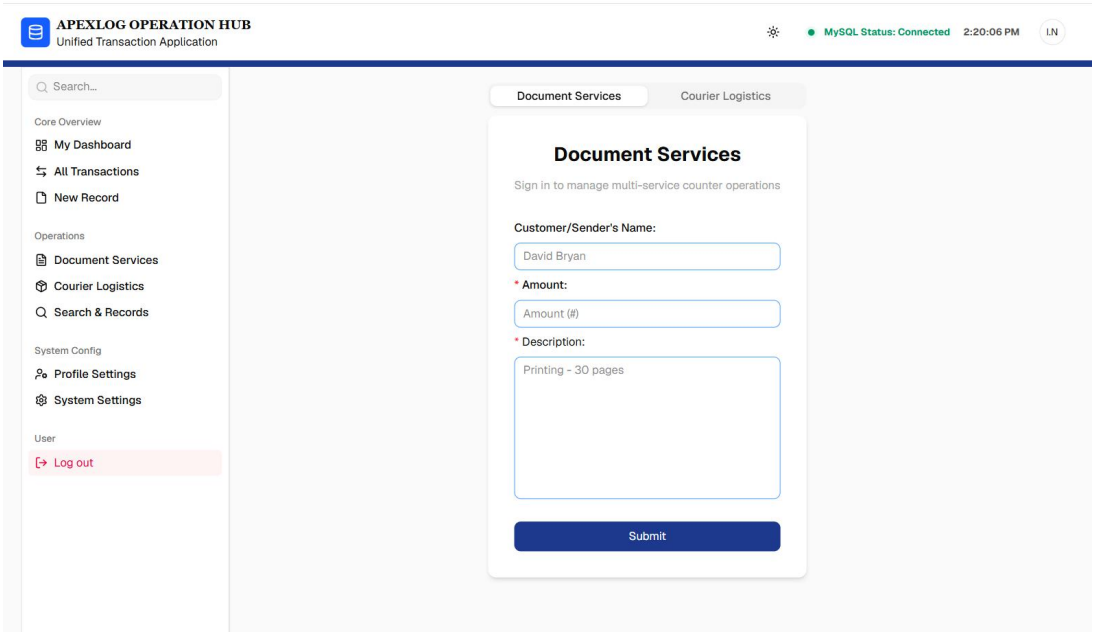

Figure 4.4.1: Document Service Transaction Entry Interface

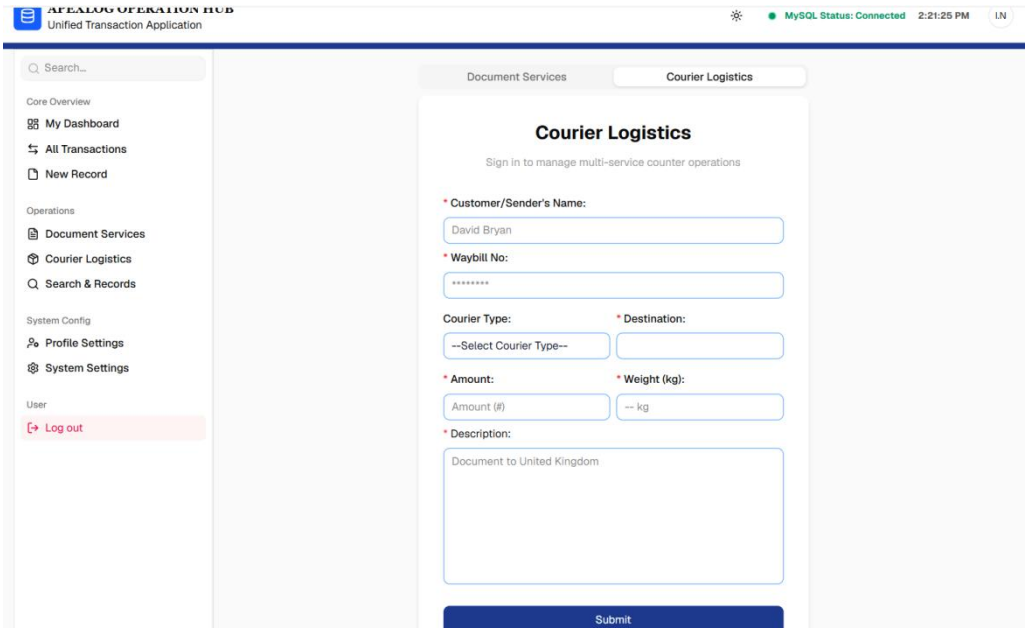

Figure 4.4.2: Courier Logistics Transaction Entry Interface

Figures 4.4.1 and **4.4.2** illustrate the system's polymorphic Transaction Logging module, which unifies distinct document processing and courier logistics workflows into a single operational interface. To maximize counter throughput, the layout is governed by a dynamic React state hook that monitors a primary service category selection. Toggling between service options alters the conditional rendering logic of the Next.js frontend, safely swapping service-specific input fields instantly without requiring a manual page refresh.

Operational Logic for Document Services

As shown in **Figure 4.4.1**, when the operator selects the Document Service workflow, the interface renders fields tailored for high-volume, quick-turnaround tasks such as printing, scanning, or typesetting. Because small counter operations often require rapid processing for transient clients, requiring a full customer profile registration for every minor job would create a severe operational bottleneck.

To resolve this issue, the system treats the customer name field as optional within this specific module. A multi-tiered defensive programming strategy handles empty fields through the following mechanisms:

- **Frontend Default Evaluation:** If the operator leaves the customer identity field blank, the Next.js data-binding layer automatically populates the payload attribute with a permanent string literal reading "Walk-In Customer" before compiling the form data.
- **Backend Fallback Validation:** To protect against situations where client-side validation scripts are intentionally bypassed or altered, the stateless PHP backend replicates this validation logic. Upon receiving the JSON payload, the API checks for a null or empty name value, automatically inserting the "Walk-In Customer" identifier before executing the MySQL insertion script.

Operational Logic for Courier Logistics

Conversely, as captured in **Figure 4.4.2**, selecting the Courier Logistics workflow triggers an entirely different set of data validation constraints. Anonymized transactions within a logistics ecosystem introduce significant legal, tracking, and operational risks.

Consequently, the system configures every single form field within this sub-module as strictly mandatory. The sender and customer name fields are designated as non-nullable variables at both the database level and the application interface layer. The Next.js client-side type guards prevent form submission entirely if any identification, destination address, or package metric field remains empty.

State Transitions, Error Mitigation, and Dashboard Synchronization

The interface utilizes high-visibility notification banners to handle state feedback during asynchronous communication with the PHP API backend:

- **Error Mitigation States:** If the database layer reports an integrity violation, or if the API backend detects missing or corrupted parameters, the application halts the submission process. It renders a prominent red alert banner positioned at the top margin of the form layout, detailing the exact error message to guide the operator.
- **Success Presentation States:** When the server confirms that data persistence has succeeded, the form renders a green success banner displaying a "Transaction Logged Successfully" message.

Immediately following a successful transaction entry, the system triggers an automated client-side routing routine. The Next.js router programmatically returns the operator to the central system dashboard. This redirection forces a reactive lifecycle update across the core analytical components.

The application refetches data from the local database, instantly updating the metrics displayed on the "Total Transactions," "Document Services," and "Courier Logistics" KPI

cards. Simultaneously, the system appends the newly created record to the top row of the "Recent Transaction" data grid, maintaining a real-time, synchronized feed of business activity across the entire application ecosystem.

4.8.5 Comprehensive Transaction Ledger and Data Retrieval

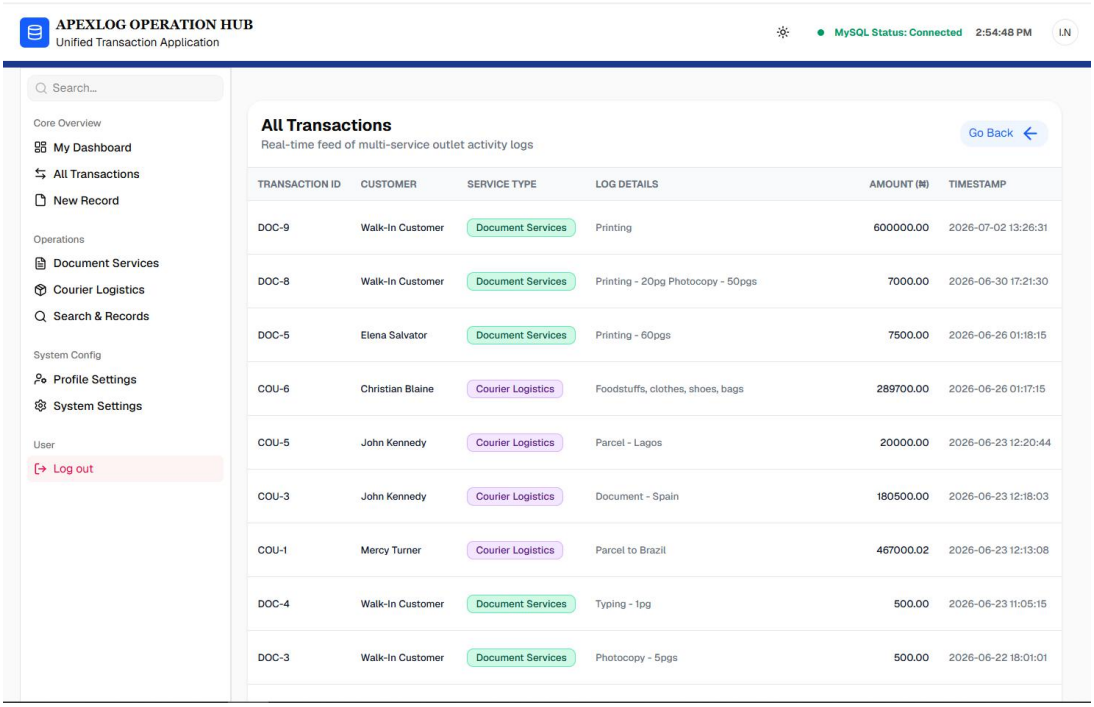

Figure 4.5: Comprehensive All Transactions Ledger Interface

Figure 4.5 illustrates the Comprehensive Transaction Ledger, which functions as the primary data retrieval, auditing, and historical tracking interface for the ApexLog system. This view provides administrative personnel with a unified, paginated data grid displaying all operational logs across both the document and courier service categories.

To optimize network latency and significantly reduce continuous server query overhead, the system architecture employs a consolidated data fetching strategy. Rather than executing isolated database requests for every page change or category filter, the PHP API backend aggregates the relational data from the MySQL tables and compiles the entire transaction history into a single, comprehensive JSON file payload. Upon mounting the "All

Transactions" component, the Next.js frontend fetches this unified JSON array just once and caches it within the local application state.

By maintaining this single JSON source of truth on the client side, the application dramatically accelerates subsequent data manipulation processes. The interface features a global search input and dynamic pagination controls located at the footer of the table. Because the entire ledger dataset resides in the browser's local memory, search queries and page navigations are executed almost instantaneously. As the operator types a query into the search bar, a client-side JavaScript filtering algorithm iterates over the JSON array, matching strings against the Transaction ID or Customer Name. This methodology updates the rendered table rows in real time without necessitating additional asynchronous HTTP requests to the backend server.

The data grid itself enforces strict structural readability. It systematically maps the JSON object attributes to standardized interface columns, including a unique Transaction ID (configured with prefixes like DOC- or COU- for quick visual distinction), Customer Name, Category, Date, operational Status, and Total Cost. To further enhance cognitive scannability, the interface utilizes color-coded visual badges to denote the transaction status, applying distinct styling to differentiate between "Completed" and "Pending" workflows.

This specific architectural approach successfully balances rigorous, persistent data tracking with the lightweight, high-speed UI rendering required to maintain efficiency in fast-paced counter operations.

4.8.6 Search and Record Retrieval Algorithm

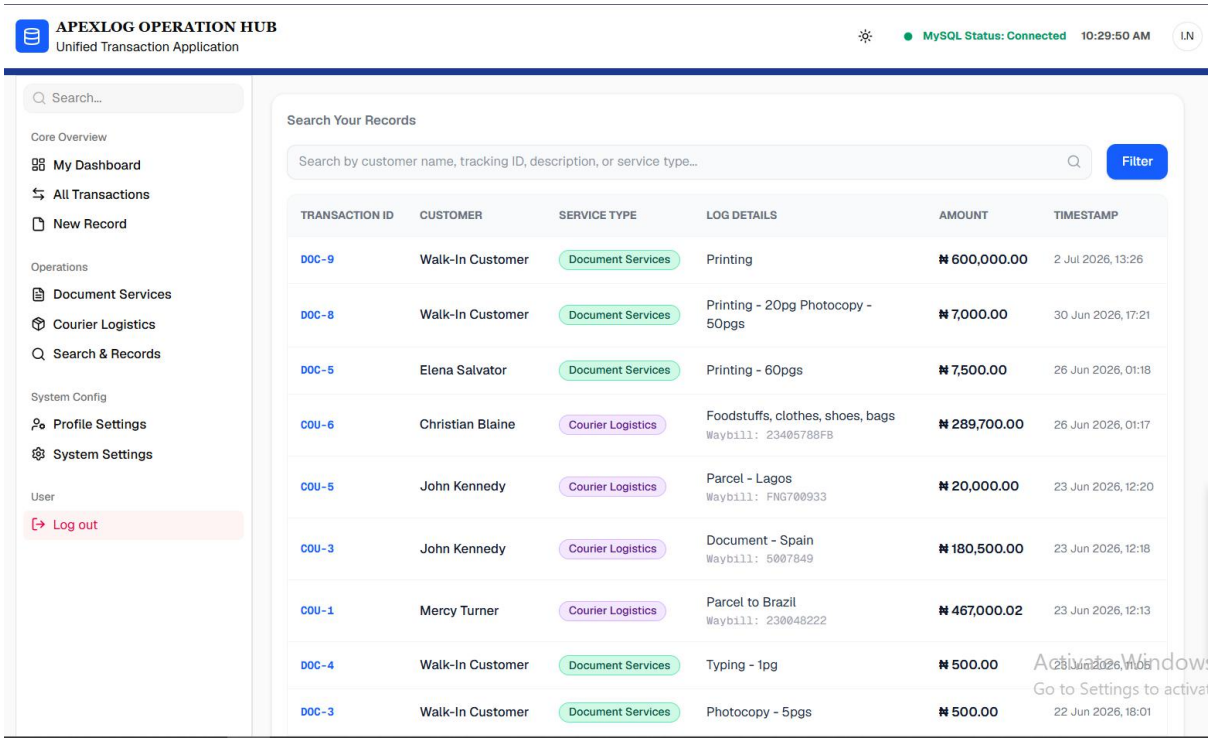

Figure 4.6: Search and Retrieval Functionality Interface

Figure 4.6 demonstrates the system's global search module, which is engineered to provide administrative operators with instantaneous access to historical transaction logs. To optimize performance and reduce continuous server load, the application utilizes a client-side filtering architecture. Upon component initialization, the Next.js frontend securely fetches the complete transaction dataset from the PHP API via an authenticated GET request. This request strictly requires the active session token to verify access rights before any data is transmitted.

Once the data payload is successfully retrieved and mounted into the local component state, the search algorithm executes in real time as the user interacts with the input field. The filtering logic utilizes case-insensitive string matching to query multiple data nodes simultaneously. Specifically, the system evaluates the user's query against the transaction ID, customer name, service type, operational description, and waybill ID. This multidimensional

scanning approach ensures that an operator can locate a specific record using any known fragment of information without requiring complex search parameters.

The user interface also dynamically formats the retrieved data for enhanced administrative readability. Financial figures are programmatically parsed into local currency format with precise decimal trailing, and timestamps are localized to the regional date and time standard. To accelerate visual parsing, conditional rendering applies distinct stylistic badges to differentiate between Document Services and Courier Logistics logs. Finally, in the event of a network disruption or server timeout, the module is equipped with a graceful error boundary that temporarily displays a diagnostic message to the user without breaking the overall interface layout.

4.8.7 Profile Management and Security Operations

The Profile Management module serves as a secure, unified interface for account administration within the ApexLog application ecosystem. Engineered as a state-driven Next.js component, the module dynamically cycles through three primary operational modes: a non-destructive viewing state, an editing interface, and a highly restricted deletion protocol. By leveraging React's conditional rendering based on a single state variable (currentView), the architecture eliminates the need for complex client-side routing between separate account pages. This approach drastically minimizes unnecessary DOM rendering and provides operators with a fluid, single-page experience for managing their credentials.

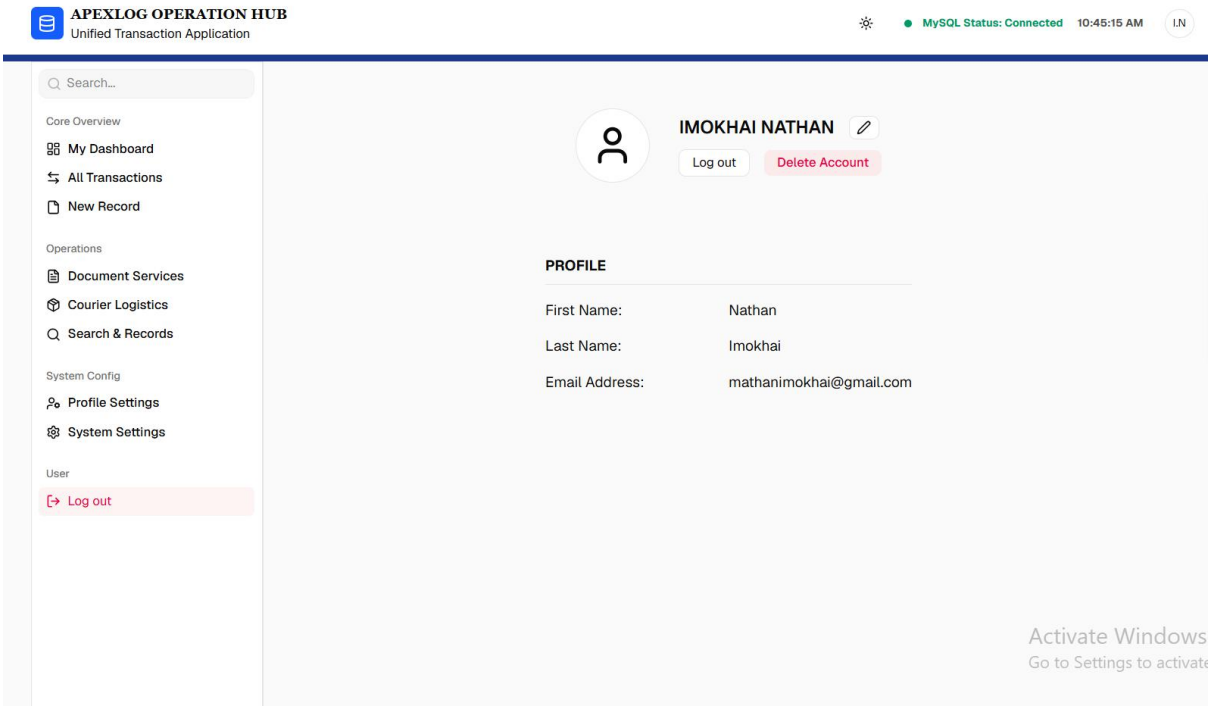

Figure 4.7.1: Profile Read-Only Interface

As illustrated in **Figure 4.7.1**, the component defaults to a read-only presentation state upon initialization, which is designed for quick verification of account details. During the mounting phase, a React useEffect lifecycle hook accesses the browser's local storage environment to retrieve the currently authenticated operator's cached identity metrics, including their first name, last name, and email address. These parameters are immediately bound to local state variables and rendered in a clean, grid-based layout beneath a structural header displaying the operator's full name in an uppercase format. This default view acts as a secure landing zone; it strictly displays current parameters without exposing interactive form inputs, preventing accidental modifications to the user's profile. Navigation to deeper administrative functions is controlled through explicit interface triggers, such as an edit icon or a designated deletion button, which programmatically transition the component's internal state.

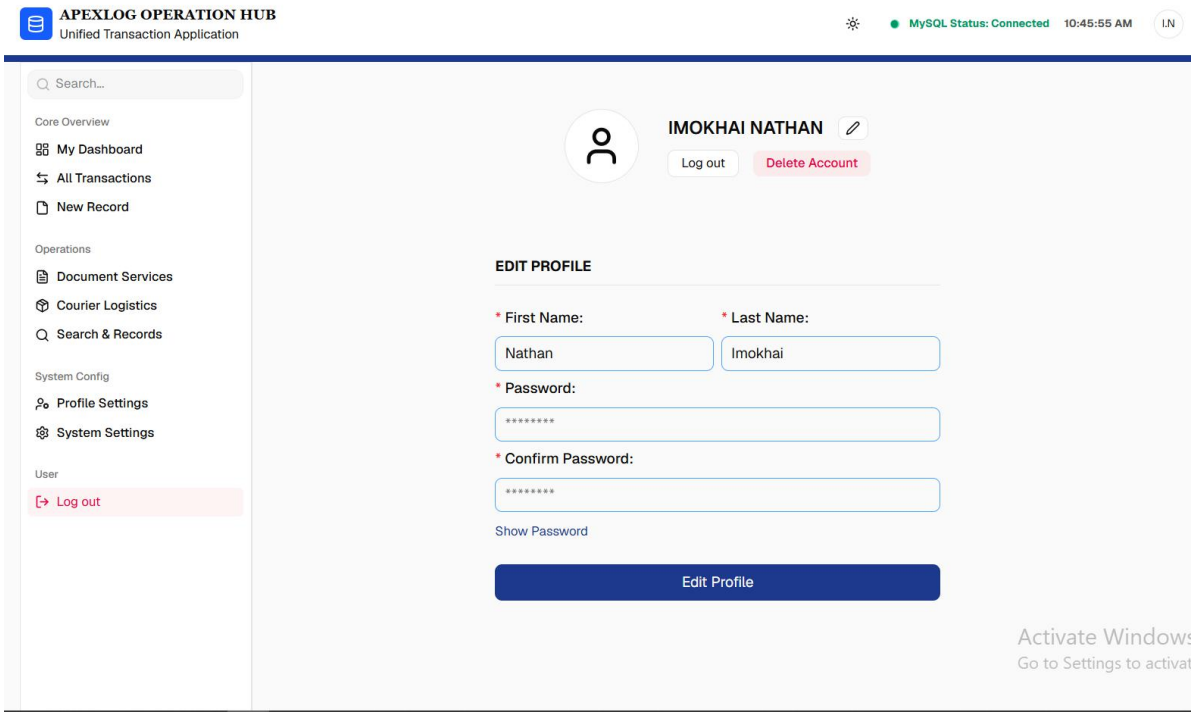

Figure 4.7.2: Profile Modification Interface

When an operator initiates an account update, the interface transitions into the modification state shown in **Figure 4.7.2**, unmounting the read-only grid and replacing it with an interactive form configuration. This state utilizes controlled inputs bound directly to a dynamic formData payload object, ensuring that every keystroke updates the underlying React state in real-time via a consolidated change handler. The interface features a dedicated toggle that dynamically modifies the input field's text visibility attribute, allowing operators to visually verify their credentials before submission. To ensure operational security, the form mandates the input and confirmation of the user's password before any mutable changes can be authorized. Upon form submission, the frontend prevents default browser reloading and executes an asynchronous network request to the dedicated backend endpoint edit-profile.php. If the server processes the update successfully, the client-side code intercepts the affirmative JSON response, instantly overwrites the local storage variables to maintain cache consistency, and displays a timed success banner before automatically returning the user to the read-only view.

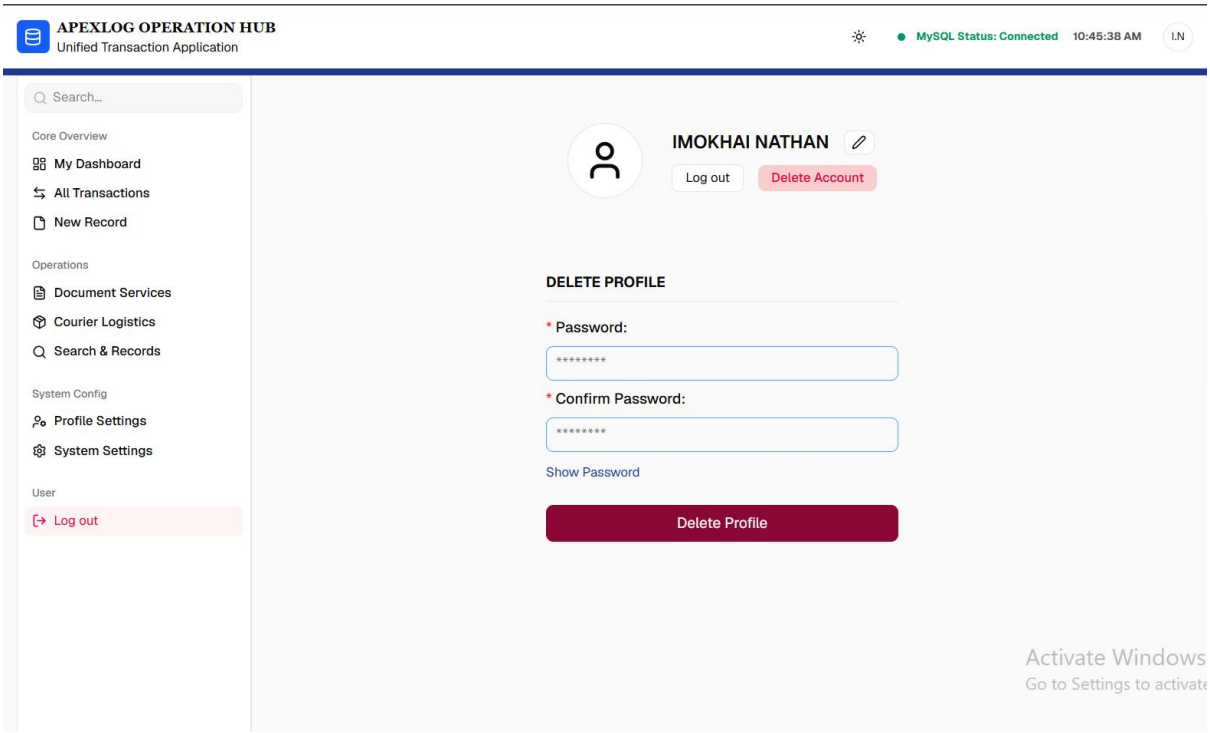

Figure 4.7.3: Account Deletion Interface

Transitioning the component to the state depicted in **Figure 4.7.3** shifts the interface into a highly restrictive, destructive confirmation mode. To protect against unauthorized or accidental account purging, the form strips away all standard data fields and narrows the interface down exclusively to password validation inputs, forcing the operator to manually re-authenticate. Submitting this form activates a dedicated asynchronous handler that dispatches the validation credentials to the backend database architecture via delete-profile.php to permanently erase the user's records. Upon receiving server-side confirmation of a successful deletion, the Next.js component initiates a comprehensive teardown of the active session. The application systematically purges all associated identification tokens, names, and emails from the browser's local storage cache to ensure zero data persistence. Once the local session is completely cleared, the application utilizes the Next.js routing engine to force an immediate redirect, returning the unauthenticated client back to the primary login gateway.

CHAPTER FIVE

CONCLUSION AND RECOMMENDATION

5.1 CONCLUSION

This research project has successfully designed, implemented, and empirically evaluated a lightweight, decoupled multi-tier transaction logging software application engineered specifically to address the operational bottlenecks, data fragmentation, and physical vulnerabilities characteristic of localized multi-service small business outlets. Historically, small-scale enterprises providing simultaneous document processing and courier logistics services have been severely constrained by an absolute reliance on manual, paper-bound logbooks and disjointed flat files. This legacy approach introduced linear lookup delays (O(N) lookup times), high clerical error rates, structural data fragmentation, and vulnerability to physical asset destruction.

By successfully fulfilling the core research objectives , this study effectively bridged the technical divide existing between complex, financially cost-prohibitive enterprise ERP platforms and resource-constrained SME outlets. The system was realized through a decoupled multi-tier web architecture pattern that structurally isolated the user presentation layer from the background backend data-persistence services.

The presentation layer was engineered utilizing a component-driven, type-safe frontend built with Next.js, TypeScript, and Tailwind CSS. This framework achieved instantaneous user interface rendering and enforced compile-time type-safety contracts, minimizing malformed inputs and significantly reducing cognitive load for operators.

Concurrently, the backend layer was developed as a stateless PHP RESTful API engine communicating with an indexed, relational MySQL database. This layer successfully centralized data processing, optimized lookup queries to sub-second thresholds via B-Tree

indexing, and eliminated data duplication across disparate service workflows by linking document and courier transactions to a single, unified customer identity.

Grounded theoretically in the Technology Acceptance Model (TAM), the empirical findings from quality assurance, latency matrix reviews, and User Acceptance Testing confirmed that the software drastically optimized both Perceived Usefulness (PU) and Perceived Ease of Use (PEOU) for end-users. The application successfully demonstrated that a low-cost, offline-capable, locally hosted Apache/XAMPP network architecture can completely eliminate transactional data loss, lower operational friction, and provide real-time, on-demand operational intelligence for small business management without necessitating expensive external cloud overhead. In conclusion, this study establishes a definitive architectural reference methodology for decoupled headless information management systems within small-scale multi-service commercial environments.

5.2 RECOMMENDATION

Based on the successful technical implementation, structural evaluations, and operational performance outcomes recorded throughout this study, the following technical and institutional recommendations are put forward:

- **Adoption of Decoupled Architecture in Local SME Environments:** It is highly recommended that localized small-scale enterprises transition completely away from manual paper ledger systems and monolithic desktop frameworks in favor of decoupled, headless web application architectures. As demonstrated in this project, isolating the frontend interface (Next.js) from the server-side logic (PHP/MySQL RESTful API) ensures low client-side computational overhead, granular stability, and high performance even when running on legacy, offline local network hardware infrastructures.

- **Standardization of Compile-Time and Static Type Verification:** Software engineering students and commercial developers targeting small business point-of-sale systems should aggressively implement static typing mechanisms using TypeScript on the presentation tier. This practice eliminates interface-to-API type mismatches, standardizes client-side form parameters, and captures structural execution anomalies during compilation before malformed payloads can be transferred to persistent database daemons.
- **Database Indexing and Parameterization for Localized Servers:** For systems operating on localized stacks such as Apache via XAMPP , developers must implement column-level B-Tree indexing on primary search attributes (such as tracking codes, customer surnames, and transaction keys). This optimization is critical to maintaining low-latency, real-time search criteria under continuous write-heavy operational data growth. Furthermore, strict use of parameterized SQL queries must be enforced as a basic architecture standard to guard against database insertion failures and SQL injection vectors.
- **Integration of Cross-Origin Security Layouts:** To preserve data perimeter integrity when running a decoupled client and API on separate ports or subdomains, future projects must integrate secure Cross-Origin Resource Sharing (CORS) header policies along with automated, time-decayed session identification rotation matrices to fully mitigate session fixation and unauthorized token access.

5.3 FUTURE SCOPE / SUGGESTIONS FOR FURTHER STUDIES

To maintain an attainable technical scope for evaluation during this project defense, certain advanced capabilities were explicitly excluded from the current build. The following areas

provide fertile grounds for subsequent software engineering iterations and future academic research:

- **Mobile Application Porting via React Native:** To build upon the current web-based single-page application dashboard, future studies should explore migrating the type-safe client components to native mobile ecosystems using frameworks like React Native. This would expand the software's accessibility parameters, allowing field dispatch personnel to interact natively with the centralized PHP database daemon directly via mobile operating devices.
- **Transition to Hybrid Cloud-Native Distributed Architecture:** While the current system successfully targets high-performance offline environments via localized XAMPP/Apache architectures, future research could detail a migration path toward hybrid cloud computing deployment patterns. This includes exploring multi-tenant distributed database clustering and real-time data sync techniques across distinct geographical counter locations.
- **Implementation of Advanced Payment Gateways and Field Encryption:** Finally, the auditing capabilities of the application can be strengthened by introducing automated financial reconciliation modules and cryptographic data integrity layers. Rather than processing front-end customer payments, future iterations could integrate read-only secure banking APIs to cross-reference recorded ledger amounts with actual bank settlements automatically. This automated auditing feature, combined with an investigation into the computational overhead of applying field-level cryptographic encryption keys to transaction records, would provide a powerful, fraud-resistant framework for small-to-medium enterprise administration.

REFERENCES

Afaishat, T. A., Al-Khaffaf, M., & Engineering Group. (2024). Impact of legacy manual records on small business scaling and operational throughput. Journal of Small Business and Enterprise Development, 31(2), 145–162.

Alade, O. J. (2023). Design and implementation of a web-based document management framework for institutional record-keeping. African Journal of Computer Science and Information Technology, 16(1), 45–58.

Astuty, W., Siregar, H., & Handoko, B. (2024). Financial and technical barriers in enterprise resource planning adoption among small-scale enterprises. International Journal of Software Engineering and Computer Systems, 10(1), 89–102.

Cherchata, L., Popova, O., & Mykhailova, I. (2022). Corporate innovations in logistics management and supply chain optimizations. Journal of Logistics and Supply Chain Management, 14(3), 204–219.

Coronel, C., & Morris, S. (2023). Database systems: Design, implementation, and management (14th ed.). Cengage Learning.

Davenport, T. H. (2021). Mission critical: Realizing the promise of enterprise systems. Harvard Business Press.

Dynamic Web Solutions. (2022). The structural shift toward integrated business software ecosystems. Tech Whitepaper Series, 4(11), 12–25.

Elmasri, R., & Navathe, S. B. (2021). Fundamentals of database systems (7th ed.). Pearson.

Fan, X., Zhang, L., & Wang, Y. (2022). A service-oriented framework for tracking user interactions in multi-layered digital ecosystems. IEEE Transactions on Services Computing, 15(4), 2110–2123.

Han, Y., Green, J., & Smith, T. (2020). Cloud-based electronic document management with fine-grained role-based access control. Computers & Security, 92, 101–115.

Hayati, N., Fitri, R., & Rahma, S. (2020). Pitfalls of paper-bound records and ledger logs in micro-business environments. Journal of Information Systems and Technology Management, 5(15), 40–52.

Hendricks, M., & Mwapwele, R. (2023). Fragmented software architectures and transactional gaps in multi-service small business hubs. African Journal of Information Systems, 15(2), 78–94.

Hofisi, C., & Chigova, P. (2023). Digital transformation strategies for mitigating office workflow bottlenecks and access latency. Journal of Administrative Sciences and Technology, 11(1), 12–27.

Ikhwan, M., & Himawati, L. (2024). Addressing the architectural gaps in commercial point-of-sale software for micro-enterprises. International Journal of Computing and Digital Systems, 15(1), 302–315.

Jordan, A., Miller, P., & Davies, R. (2022). Modern document management systems as core drivers of corporate digital transformation. International Journal of Information Management, 63, 102–114.

Laudon, K. C., & Laudon, J. P. (2021). Management information systems: Managing the digital firm (17th ed.). Pearson.

Nagrama, M. V., Santos, D. R., & Reyes, J. C. (2024). Development and evaluation of a web-based document management system for administrative offices. Global Journal of Computer Science and Technology, 24(2), 19–31.

Nel, J., & Masilela, T. (2020). Integrating automated tracking frameworks in local courier and logistics workflows. Journal of Transport and Supply Chain Management, 14(1), 1–12.

Ogunleye, O. S., Adebayo, A. O., & Bello, O. M. (2020). Transitioning from physical paper archives to electronic document management systems: Standards and structural compliance. Nigerian Journal of Computer Studies, 4(2), 85–99.

Parviainen, P., Tihinen, M., & Kääriäinen, J. (2022). Siloed software platforms versus integrated multi-service operations in digitalizing industries. Software: Practice and Experience, 52(6), 1345–1361.

Pradana, A., Putra, D., & Utama, I. (2022). Operational digitalization framework: Transitioning vulnerable paper logs into secure digital archives. Journal of Computer Science and Informatics, 18(3), 212–225.

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill.

Richardson, J., Thompson, G., & Lee, K. (2022). High-speed data persistence pipelines and execution speeds in modern web architectures. Journal of Web Engineering, 21(5), 1489–1512.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database system concepts (7th ed.). McGraw-Hill.

Sommerville, I. (2021). Software engineering (10th ed.). Pearson.

Stair, R., & Reynolds, G. (2020). Principles of information systems (14th ed.). Cengage Learning.

Triandini, E., Kabir, S., & Rahman, M. (2023). Scalability and usability challenges of enterprise ERPs in small business environments. Journal of Information Technology and Software Engineering, 13(3), 415–427.

Wibowo, A. (2023). Evolution of decentralized workplace computing: From localized spreadsheets to multi-tier web instances. Journal of Advances in Computer Networks, 11(2), 67–75.